

原材料订购与转运的分层优化模型

摘要

针对原材料供货偏差、转运损耗和两周库存目标共同作用下的生产保障问题，本文统一以产品当量描述三类材料，建立供应商边际保障评价、供应商选择与转运分配的混合整数模型，并区分稳态供货潜力与固定初始库存下的立即增产能力。

针对问题一：利用 402 家供应商 240 周的完整订货与供货记录，定义移除一家供应商后、在给定生产需求下可被保障的产品量损失，以其周均值评价重要性，列出 50 家重要供应商。S229、S361 和 S108 居前三；同时报告订货后断供率、供货规模和数量履约率，避免把无订货周直接视为供货失败。

针对问题二：在有订货周平均合同规模及历史数量履约率构成的供货模型下，至少需要 26 家供应商。即使将 25 家最大供货能力都按最佳转运损耗折算，也不足以达到每周 28200 立方米需求。最少家数条件下，24 周归一化采购成本为 49.1123 万，预计运输损耗 2469.448 立方米。

针对问题三：按增加 A 类有效供给、减少 C 类采购、控制采购费用及转运损耗的字典序优化，A 类采购增加至 20.7057 万立方米，C 类降至 6.9508 万立方米。与问题二相比运输量减少 6898.540 立方米，但采购成本略增；另给出限定于前 50 家候选的 43 家方案，呈现材料节约与供应商管理规模的取舍。

针对问题四：供货模型的稳态产能上限为 33335.925 立方米/周，提高 18.213%。若初始库存固定为原产能两周、且从首周起维持新产能两周库存，立即实施的恒定周产量为 29911.975 立方米，提高 6.071%；相应首周补库存，其后维持平衡。

独立的原始数据复算、25 家供应商上界和重排不等式构造核对主要结果；完整 24 周数据填入原附件 A、B。供货下降情景显示均值方案不能保证实际生产，另给出明确扰动界下的 48 家保护方案。所报产能与最优性均以合同供货模型为条件，不将历史均值当作已知物理极限。

关键词：供应商评价；混合整数规划；转运分配；库存约束；产能提升

一、问题重述

1.1 问题背景与已知条件

企业使用 A、B、C 三类植物纤维原材料，每年按 48 周生产，需要预先制定 24 周订购和转运方案。现有周产能为 28200 立方米产品，每立方米产品分别消耗 A、B、C 原材料 0.6、0.66、0.72 立方米；三类材料可以替代使用，但不同类别的原料体积不能直接相加为产品产量。

供应商实际供货可能高于或低于订货量，企业全部收购实际供货，并尽可能维持不少于两周生产需求的库存。8 家转运商每家每周最多运输 6000 立方米原料，通常尽量由一家转运商承运同一供应商当周的货物。A、B 的采购单价分别比 C 高 20%、10%，三类原料的运输及储存单位费用相同。题目没有给出这些费用的绝对金额和初始库存数量。

附件 1 提供 402 家供应商连续 240 周的订货量与供货量，附件 2 提供 8 家转运商对应周的运输损耗率。订货量 0 表示未订货，供货量 0 表示未供货；损耗率表中的 0 表示没有运送，不能作为零损耗观测。附件 A、B 是三个优化问题的指定结果模板。

1.2 需要完成的任务

问题一：量化 402 家供应商的供货特征，建立生产保障重要性评价模型，确定并在论文中列出 50 家最重要供应商。

问题二：参考重要性分析，确定满足生产需求至少需要多少家供应商；为其制定 24 周最经济订购方案及损耗最少的转运方案，并分析实施效果。

问题三：尽量增加 A 类、减少 C 类采购，同时降低转运损耗，重新制定订购和转运方案，并比较效果。

问题四：根据现有供应商和转运商情况，确定每周可提高的产能，并给出配套 24 周计划。问题二至四的完整订购、转运结果分别填写附件 A、B，保留文件名、原表信息和供应商顺序。

二、问题分析

四问共用原材料转化和供货—转运—库存链条：第一问描述供应商价值，第二问以生产需求约束选择合同，第三问改变材料偏好，第四问将产量本身设为决策变量。供应商排序用于解释与候选集比较，不能直接代替产能约束下的供应商组合优化。

2.1 问题一的分析

若只按供货总量排序，会忽略企业需求饱和时多余供货与短缺周供货的不同作用；若对大量指标任意赋权，又难以解释分数。本文用移除一家供应商造成的生产保障损失定义直接指标，同时保留供货规模和履约特征作为诊断。评价反映历史供货对设定需求的支持作用，不估计供应商退出后的其他厂商补偿或因果效应。

2.2 问题二的分析

至少多少家是离散组合问题，订购和转运是连续数量问题。先求最少供应商数量，再在该数量约束内比较采购成本，最后对既定供货量求最小转运损耗。初始库存未给定，必须明确其口径。本文以两周现有需求作为初始状态，并将题面的库存偏好强化为计划期末逐周库存下限；因此最终答案应连同该假设使用。

2.3 问题三的分析

A 类单价较高但单位产品耗材更低，不能仅看单价比较经济性。无转运损耗时，A 和 C 每产品立方米的采购费用均为 0.72 倍 C 类基准价，而 B 为 0.726 倍。因此增 A 减 C 主要改善原料体积和潜在运储费用，未必降低采购费。本文用明确的字典序表达偏好，并报告候选供应商规模及运费临界条件。

2.4 问题四的分析

供应上限和运输上限共同限制产品量；同时，增产使两周库存目标上升。稳态产能与首周能够立即采用的产能不一定相同。本文先求连续运输网络的最大产品当量，再由首周库存不等式推导恒定周产量上界，以配套计划验证其可达性，并对供货能力假设作敏感性分析。

三、模型假设

假设一：24 周内原料转化率、采购相对单价和运输能力保持题设值，不设置题目未给出的提前期；当周到货可用于当周生产。若存在交付延迟，库存递推需按实际到货周移动，当前计划不能原样执行。

假设二：初始库存为 56400 产品立方米当量，且生产后期末库存不低于计划周产量的两倍。原题只要求尽可能维持，本假设用于形成可计算、可核验的安全目标；若企业已有不同初始库存，应重新求解，尤其不能直接继承问题四的立即增产幅度。

假设三：供应商在有订货周的历史平均合同规模可以在未来每周执行，平均供

货随订销量线性变化，且不超过该合同规模对应的平均供给。这个范围是一种标准合同情景，不是对真实最大产能的统计识别；对于仅少数周有业务的供应商，连续供货能力尤其需要确认。

假设四：转运商历史非零损耗率的算术平均描述计划损耗；由于每周承运体积未提供，无法估计按实际运输量加权的损耗率。允许拆分一家供应商货物以满足容量和损耗目标，并明确报告拆分次数，不将软性单家承运偏好改成未说明的硬约束。

假设五：不同材料可以按产品当量共同保障生产；不假定未知的初始库存材料组成，也不计算缺乏数据支持的仓储货币成本。所有采购费用归一化到 C 类原料单价，转销量、损耗量与产品当量分别列示。

假设六：计划数量是对未来供货的模型值，不是已经实现的交付承诺。供货偏差和损耗变化另用确定性情景分析；情景倍数与边界没有被赋予实际发生概率。

四、符号说明

符号	含义	单位
i, j, t	供应商、转运商、计划周索引	无量纲
O_{iw}, S_{iw}	历史订销量、供货量	原料立方米
a_i, p_i	单位产品耗材量、相对采购单价	原料/产品体积比；相对值
$r_i, \bar{q}_i, u_i$	数量履约率、标准订销量上限、供货上限	比例；原料立方米/周
ℓ_j	非零运输周平均损耗率	比例
x_i, q_{it}, y_{ijt}	合同选择、订销量、运输量	0 或 1；原料立方米
P, I_t	计划周产量、期末库存产品当量	产品立方米
D, C^*, M_i	现有需求、稳态供给上限、边际保障量	产品立方米/周

五、模型的建立与求解

5.1 问题一：供货特征与生产保障重要性

原始数据共 402×240 组订货—供货对，没有负值或非有限数，也没有未订货却供货的记录。35881 个有订货记录中，10097 次供货为 0，约占 28.14%。因此直接令供货量等于订货量会系统忽略断供，不能支持安全库存分析。工作表尾部空白行不计为供应商。

设某历史周各供应商供货折算的产品当量之和为 H_w 。移除供应商 i 后，当周可保障产量下降的部分为 m_{iw} 。对需求截断使供过于求周的额外货量不被全额算作生产保障贡献：

$$H_w = \sum_{i=1}^{402} \frac{S_{iw}}{a_i}, \quad m_{iw} = \min(D, H_w) - \min\left(D, H_w - \frac{S_{iw}}{a_i}\right), \quad (1)$$

$$M_i = \frac{1}{240} \sum_{w=1}^{240} m_{iw}, \quad D = 28200. \quad (2)$$

该指标直接以产品立方米/周解释，不需要人为设置多个权重。它不是可相加的市场份额：每家供应商均在其他供应商保留的条件下单独移除。因此不能把各家边际值之和当作企业总产量，也不能将它解读为采购组合的唯一最优顺序。排序相同则按供应商编号保证结果可复现。

同时计算有订货周数、订货后零供货比例、平均供货量和总供货/总订货比。无订货周对重要性仍按实际供货 0 处理，但不纳入订货后断供率的分母。原料类别通过耗材系数进入产品当量，避免相同原料体积被误认为具有相同生产价值。

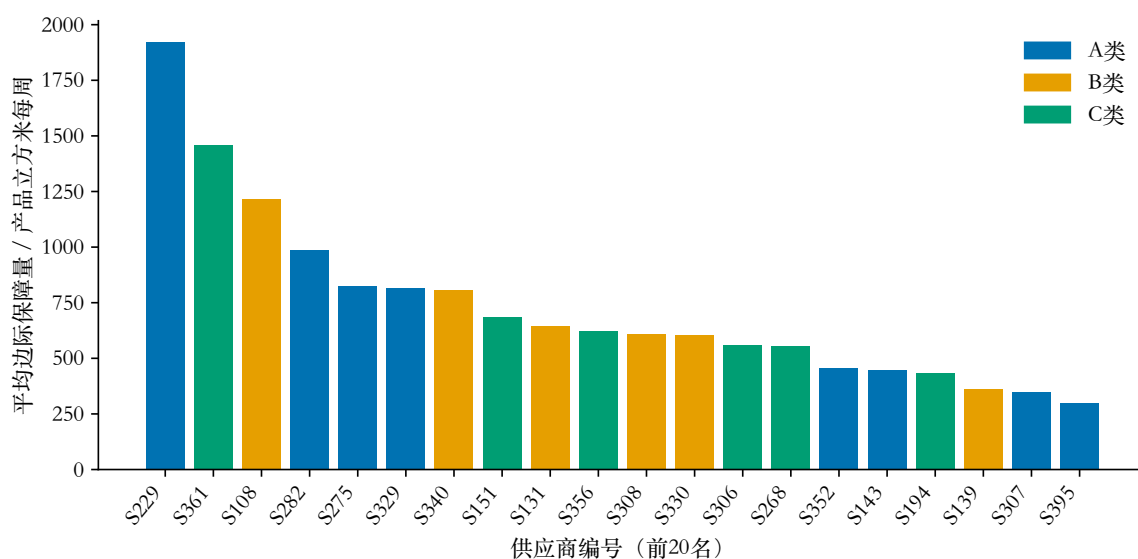


图 1 前 20 家供应商的平均边际保障量；颜色表示原材料类别。

表 1 列出完整前 50 名。与单纯按平均供货产品当量排序的前 50 名相比，有 49 家相同，说明供货规模仍是主要因素；新指标的价值在于把需求饱和与短缺周供货显式纳入解释，而不是声称复杂评分带来未经验证的预测收益。

5.1.1 50 家重要供应商名单

表 1 重要性排序；边际保障量单位为产品立方米/周。

名次	供应商	类别	边际保障量	名次	供应商	类别	边际保障量
1	S229	A	1921.47	26	S284	C	212.26
2	S361	C	1461.01	27	S031	B	187.32
3	S108	B	1216.86	28	S365	C	165.96
4	S282	A	986.55	29	S126	C	148.78
5	S275	A	826.82	30	S040	B	137.75
6	S329	A	816.08	31	S364	B	122.23
7	S340	B	808.06	32	S367	B	112.58
8	S151	C	683.68	33	S374	C	110.53
9	S131	B	645.77	34	S346	B	105.09
10	S356	C	621.82	35	S055	B	104.36
11	S308	B	610.09	36	S338	B	93.13
12	S330	B	603.44	37	S294	C	78.12
13	S306	C	561.18	38	S244	C	74.99
14	S268	C	555.53	39	S210	C	72.92

名次	供应商	类别	边际保障量	名次	供应商	类别	边际保障量
15	S352	A	455.99	40	S080	C	71.59
16	S143	A	445.48	41	S003	C	61.24
17	S194	C	432.53	42	S218	C	58.48
18	S139	B	361.50	43	S273	A	56.44
19	S307	A	349.29	44	S078	A	54.30
20	S395	A	301.16	45	S074	C	46.00
21	S140	B	298.49	46	S114	A	44.97
22	S201	A	270.69	47	S189	A	44.52
23	S348	A	258.63	48	S292	A	43.28
24	S037	C	256.68	49	S154	A	43.24
25	S247	C	237.94	50	S208	A	41.78

5.2 问题二：最少供应商与经济订购

重要性排名评估过去的生产支持，合同能力则需要解释未来可订数量。记 $\mathcal{A}_i$ 为供应商 i 有订货的历史周集合，将这些周的平均订货作为标准合同上限，以全历史供货与订货总量比描述数量履约率：

$$\bar{q}_i = \frac{\sum_{w \in \mathcal{A}_i} O_{iw}}{|\mathcal{A}_i|}, \quad r_i = \frac{\sum_w S_{iw}}{\sum_w O_{iw}}, \quad u_i = r_i \bar{q}_i = \frac{\sum_w S_{iw}}{|\mathcal{A}_i|}. \quad (3)$$

供应商的有订货周零供货已进入 r_i ；不得仅用成功交货记录估计履约率。若全年未订货，则合同上限和供货上限为 0。该线性响应在历史平均业务强度附近使用，不利用偶发最大值推出每周均可达到的能力。第 6 节另报告更激进的 90% 分位合同情景，显示能力结论对这一选择的依赖。

将材料采购单价归一化为 $p_A = 1.2, p_B = 1.1, p_C = 1$ 。每周从供应商 i 到转运商 j 的装车量为 y_{ijt} ，必须运输全部模型供货量。入库后先换算为可生产产品当量，再扣除当周生产量：

$$\sum_j y_{ijt} = r_i q_{it}, \quad 0 \leq q_{it} \leq \bar{q}_i x_i, \quad \sum_i y_{ijt} \leq 6000, \quad x_i \in \{0, 1\}, \quad (4)$$

$$I_t = I_{t-1} + \sum_{i,j} \frac{(1 - \ell_j) y_{ijt}}{a_i} - P, \quad I_0 = 56400, \quad I_t \geq 2P. \quad (5)$$

模型将库存目标施加在生产后的期末，避免把生产前有两周库存与生产后仍有两周库存混用。问题二、三取 $P = D$ 。所有装车数量按原料立方米校验 6000 上限，不能以产品当量检查原料运输能力。

求解分三个层次：先最小化签约供应商数；固定最少家数后，在所有满足该数量的组合中最小化采购费；对确定的供货量，再在生产可行的条件下最小化原料损耗。不能先固定重要性前若干家，再把其中最优选称为 402 家中的最少组合。

$$\min \sum_i x_i; \quad \min \sum_{t,i,j} p_i y_{ijt}; \quad \min \sum_{t,i,j} \ell_j y_{ijt}. \quad (6)$$

此处采购费乘的是实际模型供货量，而不是订货量；当 $r_i > 1$ 时，多供部分同样计费。采购与损耗两个目标顺序不同可能产生不同方案，本文优先最少家数和采购费用，并在这些约束下优化损耗。单位运费和仓储费用未知，不能声称已最小化包含所有未知费用的货币总成本。

由于本模型 24 周合同能力、平均损耗、需求和相对价格均不随周变化，且初始库存等于期末下限，问题二可采用每周相同的最优方案。证明是：任意 24 周可行方案的平均供货与转运量满足单周容量约束，库存终值约束要求平均入库不低于需求；线性总费用等于平均方案的 24 倍。因此在当前平稳假设下，重复单周最优方案可达到 24 周最优，不需要凭空制造周间起伏。

混合整数模型使用 HiGHS 求解，每阶段设置 90 秒上限。最少供应商求解相对间隙目标为 0，经济阶段为 10^{-7} ；本次两个阶段均终止为最优。运输子问题采用线性规划，并保留原始与对偶目标值。求解器返回状态仅是优化证据的一部分，实际运输、供货和库存另从原数据独立复算。

5.2.1 数量下界与 26 家方案

独立下界将每家供货上限按最佳转运损耗率折算为产品当量，并按大小排序。前 25 家总和只有 28044.406 立方米/周，小于 28200，因此任意 25 家都不可能满足需求；该放松忽略了最优转运商的容量，仍然不足，故是有效下界。下面的 26 家可行方案达到这个下界。

$$\sum_{k=1}^{25} \left[\frac{u_i(1 - \min_j \ell_j)}{a_i} \right]_{(k)} < D, \quad N_{\min} = 26. \quad (7)$$

表 2 给出 26 家供应商的每周订货量和预计供货量，重复用于未来 24 周。归一化采购成本为 491,123.292，模型总原料运输量 441,080.452 立方米，损耗 2,469.448 立方米。两周库存维持 56400 产品立方米当量。每周有 3 家拆分承运，其他 23 家由单一转运商承运；不声称拆分数量已全局最小。

5.2.2 每周订购和转运结果

表 2 问题二订购方案；订货和供货均为原材料立方米/周。

供应商	类别	每周订货	预计供货
S037	C	420.354	314.820
S108	B	1077.539	956.485
S126	C	1322.850	475.400
S131	B	580.754	572.967
S139	B	793.518	677.955
S140	B	2186.832	1372.941
S143	A	394.042	344.946
S151	C	1110.458	810.408
S194	C	422.417	422.354
S201	A	7116.306	1673.245
S229	A	1499.521	1478.696
S247	C	233.333	236.242
S268	C	539.042	540.775
S275	A	658.958	660.638
S282	A	702.212	705.583
S306	C	523.667	525.400
S307	A	592.924	454.628
S308	B	776.158	570.825
S329	A	651.458	652.158
S330	B	720.750	569.383
S340	B	716.667	714.275
S348	A	839.744	464.427
S352	A	372.083	370.962
S356	C	552.550	542.946
S361	C	1389.383	1367.000
S395	A	1255.190	902.893

表 3 每周各转运商的装车量与损耗率；详细分配见附件 B。

转运商	非零周数	平均损耗率 (%)	装车量 (立方米)
T1	240	1.9048	0.000
T2	240	0.9214	6000.000
T3	117	0.1861	6000.000
T4	102	1.5705	0.000
T5	83	2.8898	0.000
T6	216	0.5438	6000.000
T7	240	2.0788	0.000
T8	203	1.0103	378.352

5.3 问题三：材料结构与成本取舍

为明确‘尽量多 A、少 C’，先最大化 A 类到货对应的产品当量，再在 A 类最优值不变时最小化 C 类原料量；接着最小化采购费用，最后最小化损耗。每次优化把前一层已达到的目标值作为等式保留，不用随意大权重混合不同量纲的目标。

$$\max \sum_{t,i \in A,j} \frac{(1 - \ell_j)y_{ijt}}{0.6}, \quad \min \sum_{t,i \in C,j} y_{ijt}. \quad (8)$$

三个材料类别每产品立方米的无损采购费用分别为 $1.2 \times 0.6 = 0.72$ 、 $1.1 \times 0.66 = 0.726$ 和 $1 \times 0.72 = 0.72$ 。因此 A 单价高不代表单位产品采购一定贵，C 单价低也不代表总运储成本低。最优材料结构仍受各供应商合同能力限制，不能简单将所有订购切换为 A。

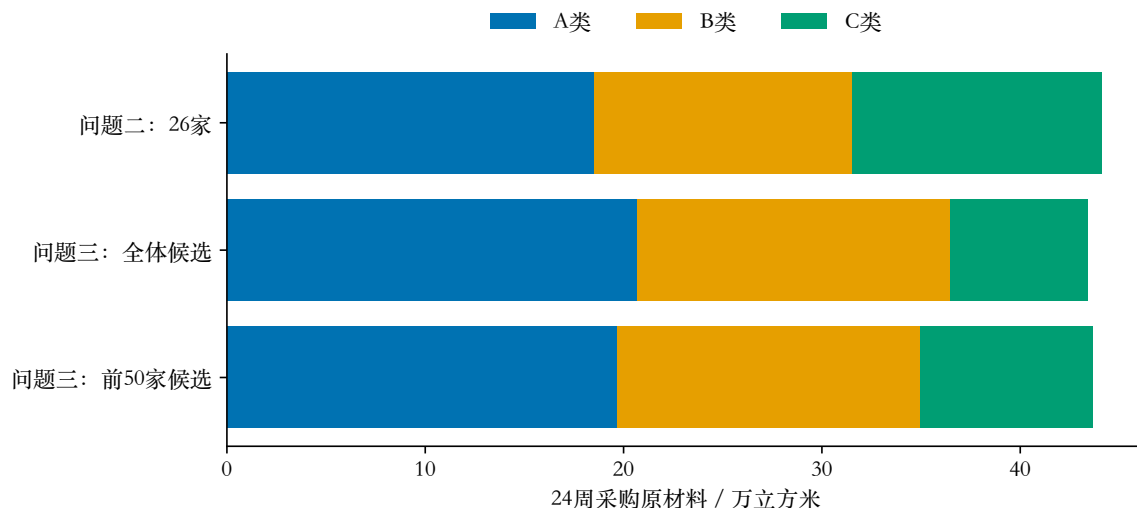


图 2 三种方案的 24 周材料采购结构；同一横轴表示原料体积，而不是产品量。

方案	供应商数	A (万立方米)	B (万立方米)	C (万立方米)	采购指数 (万)
问题二	26	18.4996	13.0436	12.5648	49.1123
问题三	310	20.7057	15.7617	6.9508	49.1355
前 50 家候选	43	19.6984	15.2667	8.7002	49.1316

问题三主方案使用 310 家供应商，采购 A 类 207,057.105 立方米、C 类 69,507.841 立方米。原料运输量比问题二少 6,898.540 立方米，预计损耗减少 69.695 立方米；归一化采购费用却增加 231.737。因此只报告‘成本下降’会混淆不同费用。

设运输单位费用为 τ ，C 类原料采购单价为 p_C 。暂不计未知的仓储和新增供应商管理费用，问题三相对问题二的已列费用下降条件为 $\tau/p_C > 0.03359$ 。若新增供应商存在固定管理费用，还应将其乘以新增 284 家计入比较。

为了给出更易组织的备选方案，将候选限制在问题一的前 50 家，再执行相同材料偏好目标，得到 43 家活动供应商。它减少合同管理规模但牺牲部分 A 类采购增量和 C 类削减效果；这是明确限制下的对照，不是证明所有 50 家以内组合的全局最优。附件 A、B 问题三页填写的是全体候选的主方案，43 家方案保存在支撑数据中，不与主表混用。

5.4 问题四：稳态潜力与立即增产

在合同能力范围内，先将周产量设为可变，以运输网络能够送达的产品当量为目标：

$$C^* = \max_{y \geq 0} \sum_{i,j} \frac{(1 - \ell_j)y_{ij}}{a_i}, \quad \sum_j y_{ij} \leq u_i, \quad \sum_i y_{ij} \leq 6000. \quad (9)$$

连续规划给出 $C^* = 33335.925$ 立方米/周，对应稳态提升 18.213%。供货能力总计 21826.520 原料立方米/周，低于 8 家总运输能力 48000；当前主瓶颈是标准合同供给，而不是总运输容量。但为了减少损耗，最佳几家转运商的 6000 上限仍会约束分配。

独立验证不重复调用线性规划。将耗材系数较小的原料先分配给损耗较低的转运商，逐一填满容量。交换较优、较劣两个配对带来的产品量增益为 $(1/a - 1/b)(\ell_{\text{bad}} - \ell_{\text{good}}) \geq 0$ ，因此按转换效率与损耗排序的构造达到重排上界。本题允许拆分，且没有禁配边，构造与线性规划一致。

若将 C^* 直接用于第一周，两周目标库存同步增加，而初始库存仍为 56400。首周最多到货 C^* ，于是期末库存要求给出另一上界：

$$I_1 = I_0 + A_1 - P \geq 2P, \quad A_1 \leq C^* \implies P \leq \frac{I_0 + C^*}{3}. \quad (10)$$

$$P^* = \min \left(C^*, \frac{I_0 + C^*}{3} \right). \quad (11)$$

代入得到 $P^* = 29911.975$ 立方米/周，即从第一周起提高 6.071%。首周按最大产品当量采购到货，补足增加的储备；其后 23 周按 P^* 维持库存。首周末库存达到 59823.950 产品立方米当量，其后不下降。该可行构造达到首周上界，完成‘上界—方案’闭环。

计划阶段	周数	每周入库 (产品立方米)	每周产量 (产品立方米)	期末库存 (产品立方米)
首周	1	33335.925	29911.975	59823.950
第 2—24 周	23	29911.975	29911.975	59823.950

若企业能在计划开始前把库存增加至两倍稳态产量，立即增产限制可以解除；若初始库存低于假设值，结论向相反方向变化。题面未给初始库存，故稳态数值与固定初始库存下的 24 周执行数值必须同时报告，不能把其中一个冒充无条件答案。

六、模型检验与灵敏度分析

6.1 预测误差与平均供货模型边界

按时间先后，使用第 1—144 周估计三个响应基线，在第 145—192 周比较，随后对第 193—240 周作诊断。基线分别为数量比率、过原点最小二乘和逐周履约率中位数。中位数在绝对误差上较小，但它估计典型供货而非平均供货；本模型涉及采购和到货总量核算，因此采用数量比率，不把中位数冒充期望。

响应基线	145—192 周有订货记录 MAE	每周总供货 MAE
总量比率	31.644	3384.702
过原点最小二乘	36.719	3655.997
履约率中位数	30.192	4019.394

数量比率在后 48 周有订货记录上的绝对误差为 38.627 原料立方米，每周总量绝对误差为 3884.641 立方米，后者相对于实际周均总供货 17828.917 约为 21.79%。这一偏差不能支持‘预测准确、供货可靠’的强结论。

探索阶段已经查看过中位数基线在该后 48 周的汇总误差，再依据估计对象不匹配修正为均值核算。因此这部分仅称按时间划分的诊断，不再包装为完全未触碰的最终测试。更重要的是，验证使用的是历史实际订货条件，不能把它当作对新增采购政策的反事实验证。未来连续供货能力仍需合同或新样本确认。

6.2 数值独立性与硬约束复核

独立复算程序重新打开原始订货、供货与损耗工作簿，不读取求解器内部约束矩阵来替代题意。它逐供应商核对供货上限和履约率，逐周逐转运商汇总装车量，并以原料类别系数重新推算入库、库存和采购费用。共 29 项检查通过，最大库存约束偏差仅为浮点舍入量。

最少家数由 25 家最佳损耗放松上界独立证明，稳态产能由重排构造独立证明；两者不是同一个求解器换参数运行。混合整数经济阶段的最优性仍依赖 HiGHS 分支定界证书，本文没有声称以另一程序穷举全部 402 家组合。基础三供应商合成算例另外用穷举核对最少家数，并检查运输瓶颈下的连续构造。

附件 A、B 保存后重新读取全部指定区域，将每个非空数值与模型数组对应核

验，零订货或零转运保留空白；六个工作表的原表信息、合并区域和顺序均保留。显示三位小数不改变底层数值，避免在大规模分配中积累手工抄录和过早舍入误差。

6.3 能力假设与初始库存敏感性

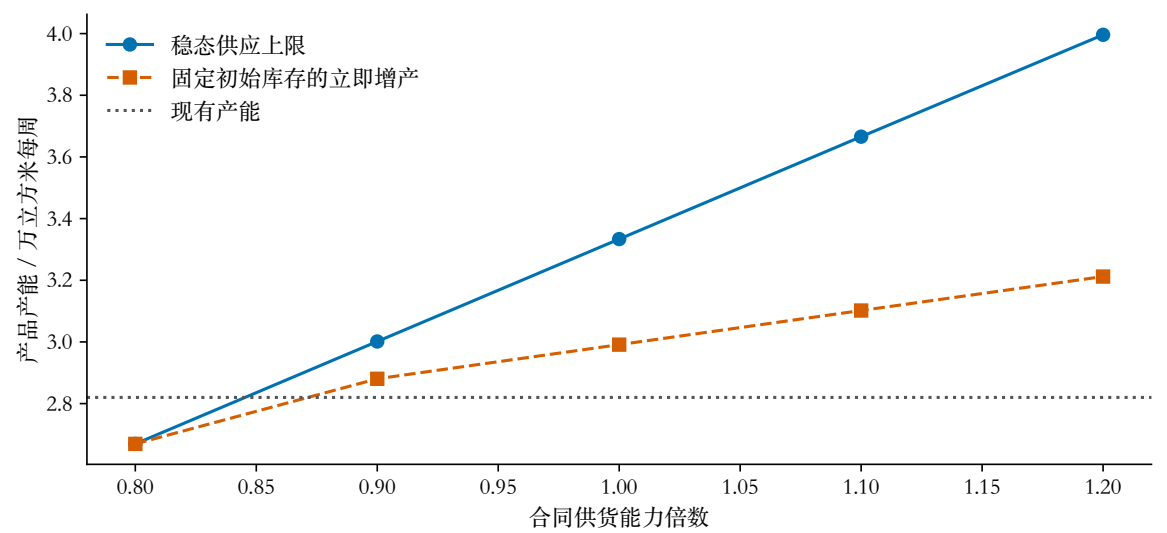


图 3 标准合同供给能力变化对稳态和立即增产的影响；初始库存固定为 56400 产品立方米当量。

供货能力倍数	稳态产能 (产品立方米每周)	立即产量 (产品立方米每周)
0.8	26693.406	26693.406
0.9	30014.998	28804.999
1.0	33335.925	29911.975
1.1	36656.781	31018.927
1.2	39960.700	32120.233

这组情景将所有标准合同供给同时缩放，直接影响可供原料的数量。在 0.8 倍情景中连现有产能都不能满足，不能再讨论增加产量。若转运损耗同时取各家历史非零周 95% 分位数，稳态产能重新求解；这个组合是较高损耗情景，不是具有 95% 联合覆盖率的统计区间。

进一步用有订货周订货量 90% 分位作为合同上限，会得到 73332.234 立方米/周的另一产能情景。这与主模型的差别揭示了仅凭历史订单识别真实产能的困难，不能因为更激进的假设给出更大数字就认为技术改造效果更好。

6.4 供货风险与保护方案

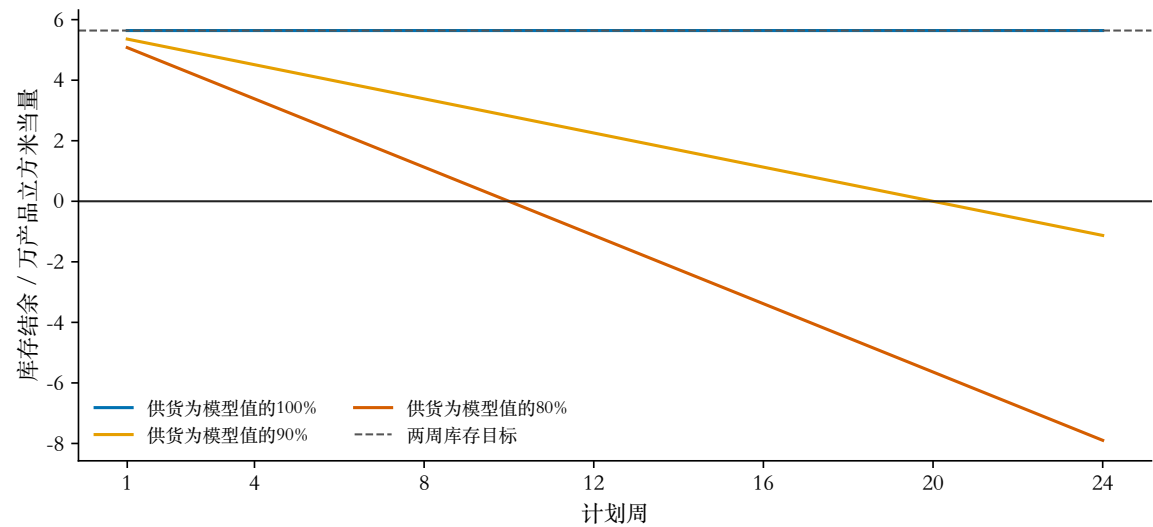


图 4 问题二在供货量下降情景下的库存结余；负值表示累计生产缺口，不表示允许出现物理负库存。

均值方案恰好补充每周消耗，持续少供会首先破坏两周储备，再耗尽库存。供货降至模型值 90% 或 80% 时，安全目标从第一周便不再满足。供货高于模型值同样不能忽略：企业仍需全部收购，原本满载的转运商可能超载，需要将多供部分改派有剩余能力的转运商。

为给出可实施的风险取舍，另设每家供货不低于均值模型 90%、各家损耗不超过其历史非零周 95% 分位的边界，按更不利的产品当量约束重新选合同。得到 48 家供应商的保护方案，其边界情景每周到货不少于 28200.000 产品立方米，所有计划装车量仍满足 6000 上限。该方案的完整数量保存在支撑数据中。

方案	供应商数	采购指数 (万)	24 周期末库存当量
均值经济方案	26	49.1123	56400.000
边界保护方案	48	55.4067	140215.735

保护方案需要更多合同、更多采购及较大的平均库存，以换取声明扰动范围内的可行性；不把这个成本隐藏在‘鲁棒’二字后面。90% 供货下界没有从合同或新观测得到保证，各转运商 95% 边界也没有联合校准，因此该方案不是 95% 可靠性保证，超出边界仍可能失败。

七、模型评价与改进

模型的优势是将三类原材料统一到产品当量，分别记录采购原料、损耗与库存；供应商最少数量和产能上限都有可检查的结构证据。字典序处理明确了采购、材料结构和损耗目标的优先级，成本临界条件揭示何时应该采用更偏 A 类的方案。完整结果由程序直接填写官方模板，便于逐周核验和调整。

主要不足来自供货信息而不是优化器精度：无订货周无法区分未被选择与无法供货，历史平均业务强度未证明能连续维持 24 周，量化诊断仍有明显误差。应优先向关键供应商确认周交付能力、停工周期和超供边界，再建立带可用性状态的滚动计划；不能靠提高求解器精度消除这些未知。

连续数量模型还没有纳入最小订单、整车批量和每家供应商的固定管理费。问题三 310 家、问题四首周 402 家活动供应商是这种简化的明显后果，实际执行应根据确认的费用比较 43 家备选和保护方案。题面运输与库存偏好也存在解释空间，若必须单家承运或初始库存不同，应相应增加整数分配约束和改写首周条件。

八、结论

第一问给出以历史边际保障量定义的重要性指标和 50 家完整名单。第二问在标准合同供给模型下最少 26 家，24 周归一化采购成本 49.1123 万，运输损耗 2469.448 立方米；数量下界与可行方案吻合。

第三问增加 A、减少 C 后总原料量下降，但采购费用略增；需结合单位运输费、仓储费和管理费判断经济性。第四问稳态供给上限为 33335.925 立方米/周，在固定初始库存和首周储备目标下立即实施的恒定产量为 29911.975 立方米/周。

结果随未来供货能力和初始状态改变。附件 A、B 给出主模型问题二至四的完整 24 周计划，支撑文件另给出 43 家材料偏好方案和 48 家边界保护方案，供企业在能力确认后选择。模型内最优和风险情景可行不等于真实生产验证。

AI 工具使用声明

本稿为赛前限时演练，由 AI 辅助完成题意解析、模型推导、编程、数值复核与中文写作，使用通用数值和排版工具。没有使用同题参赛论文或网络题解，模型预训练接触无法排除；本稿也不代表真实参赛队伍或人工专家已审核。

使用详情：AI 参与官方题面与四份附件的读取、边际保障指标构造、均值与中位数估计对象纠正、供应商混合整数规划、转运线性规划、首周库存上界推导、源数据复算、图表及结果模板生成。模型、代码与图表均由本次演练产出；未编造现

场测量、合同承诺或人工签字。正式参赛时须由真实队伍核对适用年度规则，并提供实际使用记录。

参考文献

[1] 全国大学生数学建模竞赛组委会. 2021 年高教社杯全国大学生数学建模竞赛 C 题: 生产企业原材料的订购与运输及附件 1、附件 2、附件 A、附件 B. 本次使用所提供官方原始文件.

[2] SciPy developers. `scipy.optimize.linprog` 与 `scipy.optimize.milp` 软件接口. 本次计算使用 SciPy 1.16.1 及其 HiGHS 后端; 具体求解设置和实现见源程序.

附录：支撑文件与完整源程序

以下文件与支撑包同源生成，完整代码逐文件列出。此清单不表示人工核验或正式提交已经完成。

文件	用途类别	字节数
alternatives.py	code	2679
check_workbooks.py	code	3086
figures.py	code	5040
package.py	code	3141
paper.py	code	34045
prepare.py	code	9502
renderer/package_support.py	code	5900
renderer/render_latex.py	code	13043
renderer/render_paper.py	code	7160
requirements.txt	document	110
results/alternatives.json	data	3037090
results/ capacity-sensitivity.json	data	2564
results/data-analysis.json	data	2454
results/facts.json	data	50925
results/plans.json	data	4897062
results/ranking.json	data	162679
results/verification.json	data	7702
results/ workbook-verification.json	data	2519
run_all.py	code	2782
solve.py	code	8693
test_supply.py	code	1486
verify.py	code	5827
workbooks.mjs	code	2686
使用说明.txt	document	983
附件 A 订购方案数据结果.xlsx	data	91333
附件 B 转运方案数据结果.xlsx	data	616394
manifest.json	文件校验清单	随包生成

alternatives.py

```
"""Operational tradeoffs: bounded response stress and a smaller Q3
portfolio."""
import argparse
import json
from copy import deepcopy
from pathlib import Path

import numpy as np
from prepare import write
from solve import count_suppliers, material_plan, record_plan

def alternatives(run):
    f=json.loads((run/"artifacts/facts.json").read_text());p=json.loads(
(run/"artifacts/plans.json").read_text())
    cap=np.zeros(402);idx=f["top50_indices"];cap[idx]=np.array(f["supply
_cap"])[idx]
    y,proof=material_plan(f,f["demand"],"A_then_C",cap_override=cap)
    limited=record_plan(f,np.repeat(y[None,:,:],24,axis=0),f["demand"])
    limited["scope"]="Q3 lexicographic objective restricted to the Q1
top50, not the globally best 50-supplier portfolio"
    limited["certificates"]=proof
    worst=deepcopy(f);worst["carrier_mean_loss"]=f["carrier_p95_loss"]
    y,proof=count_suppliers(worst,f["demand"]/.9)

    resilient=record_plan(f,np.repeat(y[None,:,:],24,axis=0),f["demand"])
    lower_arrivals=.9*(y*(1-np.array(f["carrier_p95_loss"]))[None,:]/np.
array(f["conversion"])[:,None]).sum()
    resilient["worst_case_weekly_arrivals"]=float(lower_arrivals)
    resilient["certificates"]=proof
    resilient["scope"]="conditional protection if every supplier
delivers at least 90% of modeled mean and each carrier loss is at most
its historical marginal 95th percentile; these joint bounds have no
calibrated confidence level"
    if lower_arrivals<f["demand"]-1e-6:raise
ValueError("bounded-response plan fails its own stress")
    saving=p["Q2"]["raw_material_volume"]-p["Q3"]["raw_material_volume"]

    increase=p["Q3"]["purchase_cost_index"]-p["Q2"]["purchase_cost_index"]
    write(run/"artifacts/alternatives.json",{
```

```

        "Q3_top50":limited,"Q2_bounded_response":resilient,
        "Q3_cost_tradeoff":{"purchase_index_increase":increase,"raw_transport_volume_saving":saving,
                            "transport_unit_fee_break_even_relative_to_C_purchase_price":increase/saving,
                            "extra_supplier_count":p["Q3"]["active_suppliers"]-p["Q2"]["active_suppliers"],
                            "scope":"ignores unknown storage and supplier administration costs; all monetary terms indexed to C raw purchase price"}})
    print({"Q3_top50_supplier_count":limited["active_suppliers"],"Q3_top50_raw_by_type":limited["raw_by_type"],
          "bounded_response_supplier_count":resilient["active_suppliers"],"bounded_response_min_arrival":lower_arrivals,
          "transport_fee_break_even":increase/saving},flush=True)

```

```

if __name__=="__main__":
    ap=argparse.ArgumentParser();ap.add_argument("run",type=Path);alternatives(ap.parse_args().run)

```

check_workbooks.py

```

"""Read exported official templates and independently reconcile every submitted cell."""

```

```

import argparse
import hashlib
import json
from pathlib import Path

```

```

import numpy as np
from openpyxl import load_workbook
from prepare import write

```

```

def check(run):
    p=json.loads((run/"artifacts/plans.json").read_text());reports=[]
    for tag in "AB":
        original=next((run/"sources").glob("* 附件"+tag+"*"))
        name=original.name[original.name.index(" 附件"):]
        result=run/"artifacts/submission"/name

```

```

        source=load_workbook(original,data_only=False);out=load_workbook
(result,data_only=False)
        assert source.sheetnames==out.sheetnames
        for q,(before,after) in enumerate(zip(source,out),2):
            width=24 if tag=="A" else 192
            expected=np.array(p["Q"+str(q)]["orders"]).T if tag=="A"
else
np.array(p["Q"+str(q)]["shipments"]).transpose(1,0,2).reshape(402,192)
            actual=np.zeros_like(expected);bad_type=[];changed=[]
            for row in
after.iter_rows(min_row=7,max_row=408,min_col=2,max_col=width+1):
                for c in row:
                    if c.value is not None:
                        if
c.data_type!="n":bad_type.append(c.coordinate)
                            actual[c.row-7,c.column-2]=float(c.value)
                for row in before.iter_rows():
                    for c in row:
                        if 7<=c.row<=408 and 2<=c.column<=width+1:continue
                        v=after.cell(c.row,c.column).value
                        # Export may normalize formula casing; formulas
remain formulas.
                            if v!=c.value:changed.append({"cell":c.coordinate,"b
efore":c.value,"after":v})
                        merges_ok=sorted(map(str,before.merged_cells.ranges))==sorte
d(map(str,after.merged_cells.ranges))
                        record={"workbook":name,"sheet":after.title,"numeric_cells":
int(np.count_nonzero(actual)),
                            "max_numeric_error":float(np.max(np.abs(expected-act
ual))), "non_numeric_output_cells":bad_type,
                            "changed_existing_information":changed, "merged_range
s_preserved":merges_ok,
                            "blank_zero_outputs":all(after.cell(i+7,j+2).value
is None for i,j in zip(*np.nonzero(expected<=1e-8)))}
                        record["passed"]=record["max_numeric_error"]<1e-7 and not
bad_type and not changed and merges_ok and record["blank_zero_outputs"]
                        reports.append(record)
                    source.close();out.close()
                report={"passed":all(r["passed"] for r in reports),"sheets":reports,

"workbook_sha256":{p.name:hashlib.sha256(p.read_bytes()).hexdigest() for
p in (run/"artifacts/submission").glob("*.xlsx")},

```

```

        "plans_sha256":hashlib.sha256((run/"artifacts/plans.json").read_bytes()).hexdigest()}
    write(run/"artifacts/workbook-verification.json",report)
    print(json.dumps(report,ensure_ascii=False,indent=2))
    if not report["passed"]:raise ValueError("exported templates differ from verified plan or source information")

```

```

if __name__=="__main__":
    ap=argparse.ArgumentParser();ap.add_argument("run",type=Path);check(
ap.parse_args().run)

```

figures.py

```

"""Chinese quantitative figures from verified supply-plan artifacts."""

```

```

import argparse
import hashlib
import json
import os
from pathlib import Path

```

```

import numpy as np
from prepare import write

```

```

def draw(run):
    out=run/"artifacts/figures";out.mkdir(parents=True,exist_ok=True)
    cache=run/"tmp/plot-cache";cache.mkdir(parents=True,exist_ok=True)
    os.environ["MPLCONFIGDIR"]=str(cache.resolve())
    import matplotlib
    matplotlib.use("Agg")
    from matplotlib import pyplot as plt,font_manager
    fontpath=Path("/System/Library/Fonts/Supplemental/Songti.ttc")
    if fontpath.exists():
        from fontTools.ttLib import TTCollection
        fonts=TTCollection(fontpath);fp=cache/"Songti.ttf";fonts.fonts[6].save(fp);fonts.close()
        font_manager.fontManager.addfont(str(fp));family=font_manager.FontProperties(fname=str(fp)).get_name()
    else:
        family=next((v.name for v in font_manager.fontManager.ttflist if v.name in ("SimSun","Noto Serif CJK SC","Source Han Serif SC")),None)

```

```

        if family is None:raise ValueError("Chinese serif font
required")
        plt.rcParams.update({"font.family":family,"font.size":11,"axes.unico
de_minus":False,"pdf.fonttype":42})
        p=json.loads((run/"artifacts/plans.json").read_text());r=json.loads(
(run/"artifacts/ranking.json").read_text())
        alt=json.loads((run/"artifacts/alternatives.json").read_text());sens
=json.loads((run/"artifacts/capacity-sensitivity.json").read_text())
        def save(fig,name):
            fig.savefig(out/(name+".pdf"),bbox_inches="tight");fig.savefig(o
ut/(name+".png"),dpi=200,bbox_inches="tight");plt.close(fig)
            fig,ax=plt.subplots(figsize=(8,4));top=r[:20]
            ax.bar(np.arange(20),[v["marginal_weekly_product_equivalent"] for v
in top],color=[{"A":"#0072B2","B":"#E69F00","C":"#009E73"}[v["kind"]]
for v in top])
            ax.set_xticks(np.arange(20),[v["supplier"] for v in
top],rotation=50,ha="right")
            ax.set_ylabel("平均边际保障量 / 产品立方米每周");ax.set_xlabel("供应商编号
(前20名) ")
            from matplotlib.patches import Patch
            ax.legend(handles=[Patch(color=c,label=k+"类") for k,c in
zip("ABC",("#0072B2","#E69F00","#009E73"))],frameon=False)
            ax.spines[["top","right"]].set_visible(False);fig.tight_layout();sav
e(fig,"supplier-importance")
            fig,ax=plt.subplots(figsize=(8,3.8));labels=["问题二: 26家","问题三:
全体候选","问题三: 前50家候选"]
            plans=[p["Q2"],p["Q3"],alt["Q3_top50"]];left=np.zeros(3)
            for k,color in zip("ABC",("#0072B2","#E69F00","#009E73")):
                v=np.array([x["raw_by_type"][k]/10000 for x in
plans]);ax.barh(labels,v,left=left,label=k+"类",color=color);left+=v
            ax.set_xlabel("24周采购原材料 /
万立方米");ax.invert_yaxis();ax.legend(ncol=3,frameon=False,loc="upper
center",bbox_to_anchor=(.5,1.16))
            ax.spines[["top","right"]].set_visible(False);fig.tight_layout();sav
e(fig,"material-tradeoff")
            fig,ax=plt.subplots(figsize=(8,3.8));rows=sens[:5]
            ax.plot([v["contract_capacity_scale"] for v in
rows],[v["steady_capacity"]/10000 for v in
rows],"o-",label="稳态供应上限",color="#0072B2")
            ax.plot([v["contract_capacity_scale"] for v in
rows],[v["immediate_capacity"]/10000 for v in
rows],"s--",label="固定初始库存的立即增产",color="#D55E00")

```

```

    ax.axhline(2.82,color="#555555",ls=":",label=" 现有产能")
    ax.set(xlabel="合同供货能力倍数",ylabel="产品产能 /
万立方米每周");ax.legend(frameon=False)
    ax.spines[["top","right"]].set_visible(False);fig.tight_layout();sav
e(fig,"capacity-inventory")
    fig,ax=plt.subplots(figsize=(8,3.8));t=np.arange(1,25);a=np.array(js
on.loads((run/"artifacts/facts.json").read_text())["conversion"])
    loss=np.array(json.loads((run/"artifacts/facts.json").read_text())["
carrier_mean_loss"])
    for m,color in ((1.,"#0072B2"),(.9,"#E69F00"),(.8,"#D55E00")):
        y=np.array(p["Q2"]["shipments"]);arr=(y*m*(1-loss)[None,None,:]/
a[None,:,None]).sum((1,2))
        inv=56400+np.cumsum(arr-28200);ax.plot(t,inv/10000,label=f"供货为
模型值的{m:.0%}",color=color)
    ax.axhline(5.64,color="#555555",ls="--",lw=1,label="两周库存目标");ax.
axhline(0,color="#222222",lw=1)
    ax.set(xlabel="计划周",ylabel="库存结余 / 万产品立方米当量",xticks=[1,4,8
,12,16,20,24]);ax.legend(frameon=False,fontsize=9,ncol=2)
    ax.spines[["top","right"]].set_visible(False);fig.tight_layout();sav
e(fig,"supply-stress")
    names=("supplier-importance","material-tradeoff","capacity-inventory
","supply-stress")
    write(out/"evidence.json",{"source_sha256":{"name:hashlib.sha256((run
/"artifacts"/name).read_bytes()).hexdigest() for name in
("plans.json","ranking.json","alternatives.json","capacity-sensitivity.j
son")}},
"figures":{"name:hashlib.sha256((out/name).read_bytes()).hexdigest() for
stem in names for name in (stem+".pdf",stem+".png")}})
    print({"figures":len(names)},flush=True)

if __name__=="__main__":
    ap=argparse.ArgumentParser();ap.add_argument("run",type=Path);draw(a
p.parse_args()).run()

```

package.py

""""Build support files and the same-source complete program appendix."""

import argparse

import json

import shutil


```

import sys
from pathlib import Path

HERE=Path(__file__).resolve().parent
COMMON=HERE/"renderer"
if not COMMON.exists():COMMON=HERE.parents[1]/"skill/cumcm-reliable-pape
r/scripts"
sys.path.insert(0,str(COMMON))
from package_support import package_support

def package(run):
    staging=run/"artifacts/support-input";staging.mkdir(parents=True,exi
st_ok=False)
    selection=[]
    names=("prepare.py","solve.py","verify.py","alternatives.py","figure
s.py","paper.py","workbooks.mjs","check_workbooks.py","run_all.py","pack
age.py","test_supply.py","requirements.txt")
    for name in names:
        target=staging/name;shutil.copy2(HERE/name,target)
        selection.append({"source":str(target.relative_to(run)),"archive
_path":name,"role":"document" if name.endswith(".txt") else
"code"})
    for name in
("render_paper.py","render_latex.py","package_support.py"):
        target=staging/"renderer"/name;target.parent.mkdir(exist_ok=True
);shutil.copy2(COMMON/name,target)
        selection.append({"source":str(target.relative_to(run)),"archive_p
ath":"renderer/"+name,"role":"code"})
    for path in sorted((run/"artifacts/submission").glob("*.xlsx")):
        selection.append({"source":str(path.relative_to(run)),"archive_p
ath":path.name,"role":"data"})
    for name in ("facts.json","plans.json","ranking.json","data-analysis
.json","verification.json","workbook-verification.json","alternatives.js
on","capacity-sensitivity.json"):
        selection.append({"source":"artifacts/"+name,"archive_path":"res
ults/"+name,"role":"data"})
    readme=staging/" 使用说明.txt"

```

readme.write_text("2021 C限时演练支撑文件\n\n安装 requirements.txt 后，运行：\npython run_all.py --source 官方原题及四份工作簿目录 --output 新输出目录\n\n默认独立复算四问数值及情景，不读取预先发布的结果来求解。原题和原始工作簿由使用者提供，文件指纹在 results/data-analysis.json。 \n加 --deliver 可重建结果工作簿、图表和包含源码附录的论文，需要本机Codex的artifact-tool Node运行时、Tectonic或XeLaTeX及中文字体。可用 --node、--artifact-modules、--latex-compiler、--cache-dir 指定已有环境；程序不自动安装。 \n\n附件A和附件B填写的是问题2、3、4主方案。43家材料结构备选与48家边界保护方案在 results/alternatives.json。归一化采购费不是已知人民币费用。能力结论依赖标准合同规模、供货响应和初始库存假设。 \n\n重新回放不改变首次限时成绩；实际队伍审核和比赛规则核验仍需真实完成。 \n",encoding="utf-8")

```
selection.append({"source":str(readme.relative_to(run)),"archive_path":readme.name,"role":"document"})
receipt=package_support(run,{"files":selection})
print({"support_files":receipt["file_count"],"complete_source_files":receipt["code_count"],"sha256":receipt["sha256"]},flush=True)
```

```
if __name__=="__main__":
    ap=argparse.ArgumentParser();ap.add_argument("run",type=Path);package(ap.parse_args()).run()
```

paper.py

```
"""Compose a complete four-question Chinese paper from verified numerical artifacts."""
```

```
import argparse
import hashlib
import json
from pathlib import Path
```

```
import numpy as np
from prepare import write
```

```
def compose(run):
    load=lambda name:json.loads((run/"artifacts"/name).read_text())
    f=load("facts.json");p=load("plans.json");ranking=load("ranking.json");data=load("data-analysis.json")
    v=load("verification.json");alt=load("alternatives.json");sensitivity=load("capacity-sensitivity.json")
```

```

    if not v["passed"] or v["plans_sha256"]!=hashlib.sha256((run/"artifacts/plans.json").read_bytes()).hexdigest():
        raise ValueError("paper requires current independently checked plans")
    q2,q3,q4=p["Q2"],p["Q3"],p["Q4"];sections=[]
    text=lambda s:{"text":s}
    eq=lambda s:{"equation":s}
    def section(title,blocks,level=1,page=False):
        sections.append({"title":title,"blocks":blocks,"level":level,"page_break_before":page})
    def figure(stem,caption):return {"image":f"artifacts/figures/{stem}.pdf","width_cm":15.5,"caption":caption}
    fmt=lambda x:f"{x:,.3f}"
    section(" 摘要",[
        text("针对原材料供货偏差、转运损耗和两周库存目标共同作用下的生产保障问题，
        本文统一以产品当量描述三类材料，建立供应商边际保障评价、
        供应商选择与转运分配的混合整数模型，并区分稳态供货潜力与固定初始库存下的立即增产能力。
        ")),
        {"lead": "针对问题一：","text": "利用402家供应商240周的完整订货与供货记录，
        定义移除一家供应商后、在给定生产需求下可被保障的产品量损失，以其周均值评价重要性，
        列出50家重要供应商。S229、S361和S108居前三；同时报告订货后断供率、
        供货规模和数量履约率，避免把无订货周直接视为供货失败。"},
        {"lead": "针对问题二：",
        "text": f"在有订货周平均合同规模及历史数量履约率构成的供货模型下，
        至少需要{q2['active_suppliers']}家供应商。
        即使将25家最大供货能力都按最佳转运损耗折算，也不足以达到每周28200立方米需求。
        最少家数条件下，24周归一化采购成本为{q2['purchase_cost_index']/10000:.4f}万，
        预计运输损耗{q2['expected_transport_loss']:.3f}立方米。"},
        {"lead": "针对问题三：","text": f"按增加A类有效供给、减少C类采购、
        控制采购费用及转运损耗的字典序优化，
        A类采购增加至{q3['raw_by_type']['A']/10000:.4f}万立方米，
        C类降至{q3['raw_by_type']['C']/10000:.4f}万立方米。与问题二相比运输量减少{alt['Q3_cost_tradeoff']['raw_transport_volume_saving']:.3f}立方米，但采购成本略增；
        另给出限定于前50家候选的43家方案，呈现材料节约与供应商管理规模的取舍。"},
        {"lead": "针对问题四：",
        "text": f"供货模型的稳态产能上限为{q4['steady_capacity']:.3f}立方米/周，
        提高{q4['steady_increase_percent']:.3f}%。若初始库存固定为原产能两周、
        且从首周起维持新产能两周库存，
        立即实施的恒定周产量为{q4['production_per_week']:.3f}立方米，
        提高{q4['immediate_increase_percent']:.3f}%；相应首周补库存，其后维持平衡。"},

```

```

text("独立的原始数据复算、25家供应商上界和重排不等式构造核对主要结果；
完整24周数据填入原附件A、B。供货下降情景显示均值方案不能保证实际生产，
另给出明确扰动界下的48家保护方案。所报产能与最优性均以合同供货模型为条件，
不将历史均值当作已知物理极限。"),
text(" 关键词：供应商评价；混合整数规划；转运分配；库存约束；产能提升"))
section(" 一、问题重述",[],page=True)
section("1.1 问题背景与已知条件",[
text("企业使用A、B、C三类植物纤维原材料，每年按48周生产，
需要预先制定24周订购和转运方案。现有周产能为28200立方米产品，每立方米产品分别消耗A、B、
C原材料0.6、0.66、0.72立方米；三类材料可以替代使用，
但不同类别的原料体积不能直接相加为产品产量。"),
text("供应商实际供货可能高于或低于订货量，企业全部收购实际供货，
并尽可能维持不少于两周生产需求的库存。8家转运商每家每周最多运输6000立方米原料，
通常尽量由一家转运商承运同一供应商当周的货物。A、B的采购单价分别比C高20%、10%，
三类原料的运输及储存单位费用相同。题目没有给出这些费用的绝对金额和初始库存数量。"),
text("附件1提供402家供应商连续240周的订货量与供货量，
附件2提供8家转运商对应周的运输损耗率。订货量0表示未订货，供货量0表示未供货；
损耗率表中的0表示没有运送，不能作为零损耗观测。附件A、B是三个优化问题的指定结果模板。
")],2)
section("1.2 需要完成的任务",[
text("问题一：量化402家供应商的供货特征，建立生产保障重要性评价模型，
确定并在论文中列出50家最重要供应商。"),
text("问题二：参考重要性分析，确定满足生产需求至少需要多少家供应商；
为其制定24周最经济订购方案及损耗最少的转运方案，并分析实施效果。"),
text("问题三：尽量增加A类、减少C类采购，同时降低转运损耗，
重新制定订购和转运方案，并比较效果。"),
text("问题四：根据现有供应商和转运商情况，确定每周可提高的产能，
并给出配套24周计划。问题二至四的完整订购、转运结果分别填写附件A、B，保留文件名、
原表信息和供应商顺序。")],2)
section(" 二、问题分析",[
text("四问共用原材料转化和供货—转运—库存链条：第一问描述供应商价值，
第二问以生产需求约束选择合同，第三问改变材料偏好，第四问将产量本身设为决策变量。
供应商排序用于解释与候选集比较，不能直接代替产能约束下的供应商组合优化。"))
analyses=[
("2.1 问题一的分析","若只按供货总量排序，
会忽略企业需求饱和时多余供货与短缺周供货的不同作用；若对大量指标任意赋权，
又难以解释分数。本文用移除一家供应商造成的生产保障损失定义直接指标，
同时保留供货规模和履约特征作为诊断。评价反映历史供货对设定需求的支持作用，
不估计供应商退出后的其他厂商补偿或因果效应。"),

```

("2.2 问题二的分析","至少多少家是离散组合问题,订购和转运是连续数量问题。先求最少供应商数量,再在该数量约束内比较采购成本,最后对既定供货量求最小转运损耗。初始库存未给定,必须明确其口径。本文以两周现有需求作为初始状态,并将题面的库存偏好强化为计划期末逐周库存下限;因此最终答案应连同该假设使用。"),

("2.3 问题三的分析","A类单价较高但单位产品耗材更低,不能仅看单价比较经济性。无转运损耗时,A和C每产品立方米的采购费用均为0.72倍C类基准价,而B为0.726倍。因此增A减C主要改善原料体积和潜在运储费用,未必降低采购费。本文用明确的字典序表达偏好,并报告候选供应商规模及运费临界条件。"),

("2.4 问题四的分析","供应上限和运输上限共同限制产品量;同时,增产使两周库存目标上升。稳态产能与首周能够立即采用的产能不一定相同。本文先求连续运输网络的最大产品当量,再由首周库存不等式推导恒定周产量上界,以配套计划验证其可达性,并对供货能力假设作敏感性分析。")]

```
for title,body in analyses:section(title,[text(body)],2)
section(" 三、模型假设",[
    text("假设一: 24周内原料转化率、采购相对单价和运输能力保持题设值,
    不设置题目未给出的提前期;当周到货可用于当周生产。若存在交付延迟,
    库存递推需按实际到货周移动,当前计划不能原样执行。"),
    text("假设二: 初始库存为56400产品立方米当量,
    且生产后期末库存不低于计划周产量的两倍。原题只要求尽可能维持,本假设用于形成可计算、
    可核验的安全目标;若企业已有不同初始库存,应重新求解,
    尤其不能直接继承问题四的立即增产幅度。"),
    text("假设三: 供应商在有订货周的历史平均合同规模可以在未来每周执行,
    平均供货随订货量线性变化,且不超过该合同规模对应的平均供给。这个范围是一种标准合同情景,
    不是对真实最大产能的统计识别;对于仅少数周有业务的供应商,连续供货能力尤其需要确认。"),
    text("假设四: 转运商历史非零损耗率的算术平均描述计划损耗;
    由于各周承运体积未提供,无法估计按实际运输量加权的损耗率。
    允许拆分一家供应商货物以满足容量和损耗目标,并明确报告拆分次数,
    不将软性单家承运偏好改成未说明的硬约束。"),
    text("假设五: 不同材料可以按产品当量共同保障生产;
    不假定未知的初始库存材料组成,也不计算缺乏数据支持的仓储货币成本。
    所有采购费用归一化到C类原料单价,转运量、损耗量与产品当量分别列示。"),
    text("假设六: 计划数量是对未来供货的模型值,不是已经实现的交付承诺。
    供货偏差和损耗变化另用确定性情景分析;情景倍数与边界没有被赋予实际发生概率。"))
section(" 四、符号说明",[{"table":[{" 符号"," 含义"," 单位"],
    [{" $i,j,t$ "," 供应商、转运商、计划周索引"," 无量纲"],
    [{" $0_{iw},S_{iw}$ "," 历史订货量、供货量"," 原料立方米"],
    [{" $a_{i,p_i}$ "," 单位产品耗材量、相对采购单价"," 原料/产品体积比;相对值"],
    [{" $r_{i,\bar{q}_i,u_i}$ "," 数量履约率、标准订货上限、供货上限"," 比例;
    原料立方米/周"],
    [{" $v_{ell_j}$ "," 非零运输周平均损耗率"," 比例"],
    [{" $x_{i,q_{it}},y_{ijt}$ "," 合同选择、订货量、运输量"," 0或1;原料立方米"],
    [{" $P,I_t$ "," 计划周产量、期末库存产品当量"," 产品立方米"]}]}
```

```

["$D,C^*,M_i$", "现有需求、稳态供给上限、边际保障量", "产品立方米/周"]])
section(" 五、模型的建立与求解", [])
section("5.1 问题一：供货特征与生产保障重要性", [
    text(f"原始数据共402×240组订货—供货对，没有负值或非有限数，
    也没有未订货却供货的记录。{data['positive_orders']}个有订货记录中，
    {data['zero_supply_given_order_count']}次供货为0，约占{100*data['zero_supply_given_order_count']/data['positive_orders']:.2f}%。
    因此直接令供货量等于订货量会系统忽略断供，不能支持安全库存分析。
    工作表尾部空自行不计为供应商。"),
    text("设某历史周各供应商供货折算的产品当量之和为$H_w$。移除供应商$i$后，
    当周可保障产量下降的部分为$m_{iw}$。
    对需求截断使供过于求周的额外货量不被全额算作生产保障贡献："),
    eq(r"H_w=\sum_{i=1}^{402}\frac{S_{iw}}{a_i},\quad m_{iw}=\min(D,H_w)-\min\left(D,H_w-\frac{S_{iw}}{a_i}\right),",
    eq(r"M_i=\frac{1}{240}\sum_{w=1}^{240}m_{iw},\quad D=28200."),
    text("该指标直接以产品立方米/周解释，不需要人为设置多个权重。
    它不是可相加的市场份额：每家供应商均在其他供应商保留的条件下单独移除。
    因此不能把各家边际值之和当作企业总产量，也不能将它解读为采购组合的唯一最优顺序。
    排序相同则按供应商编号保证结果可复现。"),
    text("同时计算有订货周数、订货后零供货比例、平均供货量和总供货/总订货比。
    无订货周对重要性仍按实际供货0处理，但不纳入订货后断供率的分母。
    原料类别通过耗材系数进入产品当量，避免相同原料体积被误认为具有相同生产价值。"),
    figure("supplier-importance", "图1 前20家供应商的平均边际保障量；
    颜色表示原材料类别。"),
    text(f"表1列出完整前50名。与单纯按平均供货产品当量排序的前50名相比，
    有{data['top50_overlap_with_mean_supply_equivalent']}家相同，
    说明供货规模仍是主要因素：新指标的价值在于把需求饱和与短缺周供货显式纳入解释，
    而不是声称复杂评分带来未经验证的预测收益。"), 2)
    table=[["名次", "供应商", "类别", "边际保障量", "名次", "供应商", "类别", "边际保障量"]]
    for i in range(25):
        row=[]
        for z in
(ranking[i], ranking[i+25]): row.extend([str(z["rank"]), z["supplier"], z["kind"], f"{z['marginal_weekly_product_equivalent']:.2f}"])
        table.append(row)
    section("5.1.1
50家重要供应商名单", [{"table": table, "long_table": True, "caption": "表1
重要性排序；边际保障量单位为产品立方米/周。"}], 3)
    section("5.2 问题二：最少供应商与经济订购", [

```

text(r"重要性排名评估过去的生产支持，合同能力则需要解释未来可订数量。
记 $\mathcal{A}_i$ 为供应商*i*有订货的历史周集合，将这些周的平均订货作为标准合同上限，
以全历史供货与订货总量比描述数量履约率："),

eq(r"\bar{q}_i=\frac{\sum_{w\in\mathcal{A}_i}0_{\{iw\}}}{|\mathcal{A}_i|},\quad r_i=\frac{\sum_wS_{\{iw\}}}{\sum_w0_{\{iw\}}},\quad u_i=r_i\bar{q}_i=\frac{\sum_wS_{\{iw\}}}{|\mathcal{A}_i|}."),

text("供应商的有订货周零供货已进入 r_i ；不得仅用成功交货记录估计履约率。
若全年未订货，则合同上限和供货上限为0。该线性响应在历史平均业务强度附近使用，
不利用偶发最大值推出每周均可达到的能力。第6节另报告更激进的90%分位合同情景，
显示能力结论对这一选择的依赖。"),

text("将材料采购单价归一化为 $p_A=1.2, p_B=1.1, p_C=1$ 。
每周从供应商*i*到转运商*j*的装车量为 $y_{\{ijt\}}$ ，必须运输全部模型供货量。
入库后先换算为可生产产品当量，再扣除当周生产量："),

eq(r"\sum_{jy_{\{ijt\}}=r_{iq_{\{it\}}},\quad 0\leq q_{\{it\}}\leq\bar{q}_{ix_i},\quad \sum_i y_{\{ijt\}}\leq 6000,\quad x_i\in\{0,1\}."),

eq(r"I_t=I_{t-1}+\sum_{i,j}\frac{(1-\ell_j)y_{\{ijt\}}}{a_i}-P,\quad I_0=56400,\quad I_t\geq 2P."),

text("模型将库存目标施加在生产后的期末，
避免把生产前有两周库存与生产后仍有两周库存混用。问题二、三取 $P=D$ 。
所有装车数量按原料立方米校验6000上限，不能以产品当量检查原料运输能力。"),

text("求解分三个层次：先最小化签约供应商数；固定最少家数后，
在所有满足该数量的组合中最小化采购费；对确定的供货量，
再在生产可行的条件下最小化原料损耗。不能先固定重要性前若干家，
再把其中最优误称为402家中的最少组合。"),

eq(r"\min\sum_i x_i;\quad \min\sum_{t,i,j}p_{iy_{\{ijt\}};\quad \min\sum_{t,i,j}\ell_jy_{\{ijt\}}."),

text("此处采购费乘的是实际模型供货量，而不是订货量；当 $r_i>1$ 时，
多供部分同样计费。采购与损耗两个目标顺序不同可能产生不同方案。
本文优先最少家数和采购费用，并在这些约束下优化损耗。单位运费和仓储费用未知，
不能声称已最小化包含所有未知费用的货币总成本。"),

text("由于本模型24周合同能力、平均损耗、需求和相对价格均不随周变化，
且初始库存等于期末下限，问题二可采用每周相同的最优方案。证明是：
任意24周可行方案的平均供货与转运量满足单周容量约束，库存终值约束要求平均入库不低于需求；
线性总费用等于平均方案的24倍。因此在当前平稳假设下，重复单周最优方案可达到24周最优，
不需要凭空制造周间起伏。"),

text("混合整数模型使用HiGHS求解，每阶段设置90秒上限。
最少供应商求解相对间隙目标为0，经济阶段为 10^{-7} ；本次两个阶段均终止为最优。
运输子问题采用线性规划，并保留原始与对偶目标值。求解器返回状态仅是优化证据的一部分，
实际运输、供货和库存另从原数据独立复算。"))],2)

cert=q2["certificates"]["supplier_selection"]
section("5.2.1 数量下界与 26 家方案",[

text(f"独立下界将每家供货上限按最佳转运损耗率折算为产品当量，并按大小排序。
前25家总和只有{v['minimum_count_proof']['best_possible_output_with_n_minus_one']:.3f}立方米/周，小于28200，因此任意25家都不可能满足需求；
该放松忽略了最优转运商的容量，仍然不足，故是有效下界。下面的26家可行方案达到这个下界。
"),

eq(r"\sum_{k=1}^{25}\left[\frac{u_i(1-\min_j\ell_j)}{a_i}\right]_{(k)}<D,\quad N_{\min}=26."),

text(f"表2给出26家供应商的每周订货量和预计供货量，重复用于未来24周。
归一化采购成本为{fmt(q2['purchase_cost_index'])}，
模型总原料运输量{fmt(q2['raw_material_volume'])}立方米，
损耗{fmt(q2['expected_transport_loss'])}立方米。
两周库存维持56400产品立方米当量。每周有3家拆分承运，其他23家由单一转运商承运；
不声称拆分数已全局最小。"),3)

ids=[i for i,q in enumerate(q2["orders"][0]) if q>1e-6]
section("5.2.2 每周订购和转运结果",[

{"table": [{"供应商", "类别", "每周订货", "预计供货"] + [[f["ids"][i], f["kinds"][i], f"{q2['orders'][0][i]:.3f}", f"{sum(q2['shipments'][0][i]):.3f}"] for i in ids],

"long_table": True, "caption": "表2 问题二订购方案；
订货和供货均为原材料立方米/周。"},

{"table": [{"转运商", "非零周数", "平均损耗率/%", "装车量/立方米"]} + [[f"T{j+1}", str(240-data['loss_zero_missing_counts'][j]), f"{100*f['carrier_mean_loss'][j]:.4f}", f"{np.array(q2['shipments'])[0,:,j].sum():.3f}"] for j in range(8)],

"caption": "表3 每周各转运商的装车量与损耗率；详细分配见附件B。
"}],3,page=True)

section("5.3 问题三：材料结构与成本取舍",[

text("为明确‘尽量多A、少C’，先最大化A类到货对应的产品当量，
再在A类最优值不变时最小化C类原料量；接着最小化采购费用，最后最小化损耗。
每次优化把前一层已达到的目标值作为等式保留，不用随意大权重混合不同量纲的目标。"),

eq(r"\max\sum_{t,i\in A,j}\frac{(1-\ell_j)y_{ijt}}{0.6},\quad \min\sum_{t,i\in C,j}y_{ijt}."),

text(r"三个材料类别每产品立方米的无损采购费用分别为\$1.2\times 0.6=0.72\$、
\$1.1\times 0.66=0.726\$和\$1\times 0.72=0.72\$。因此A单价高不代表单位产品采购一定贵，
C单价低也不代表总运储成本低。最优材料结构仍受各供应商合同能力限制，
不能简单将所有订购切换为A。"),

figure("material-tradeoff","图2 三种方案的24周材料采购结构；
同一横轴表示原料体积，而不是产品量。"),

`{"table": [{"方案", "供应商数", "A/万立方米", "B/万立方米", "C/万立方米", "采购指数/万"}] + [[label, str(x['active_suppliers']), *["{x['raw_by_type'] [k] / 10000:.4f}" for k in "ABC"], f"{x['purchase_cost_index'] / 10000:.4f}"] for label, x in ((("问题二", q2), ("问题三", q3), ("前50家候选", alt['Q3_top50'])))]},`
`text(f"问题三主方案使用{q3['active_suppliers']}家供应商，`
`采购A类{fmt(q3['raw_by_type'] ['A'])}立方米、`
`C类{fmt(q3['raw_by_type'] ['C'])}立方米。原料运输量比问题二少{fmt(alt['Q3_cost_tradeoff'] ['raw_transport_volume_saving'])}立方米，`
`预计损耗减少{q2['expected_transport_loss'] - q3['expected_transport_loss']:.3f}立方米；`
`归一化采购费用却增加{alt['Q3_cost_tradeoff'] ['purchase_index_increase']:.3f}`
`。因此只报告‘成本下降’会混淆不同费用。`
`"），`

`text(fr"设运输单位费用为 $\tau$ ，C类原料采购单价为 $p_C$ 。`
`暂不计未知的仓储和新增供应商管理费用，`
`问题三相对问题二的已列费用下降条件为 $\tau/p_C > \{alt['Q3\_cost\_tradeoff'] ['transport\_unit\_fee\_break\_even\_relative\_to\_C\_purchase\_price']:.5f\}$ 。`
`若新增供应商存在固定管理费用，`
`还应将其乘以新增{alt['Q3_cost_tradeoff'] ['extra_supplier_count']}家计入比较。`
`"），`

`text("为了给出更易组织的备选方案，将候选限制在问题一的前50家，`
`再执行相同材料偏好目标，得到43家活动供应商。`
`它减少合同管理规模但牺牲部分A类采购增量和C类削减效果；这是明确限制下的对照，`
`不是证明所有50家以内组合的全局最优。附件A、B问题三页填写的是全体候选的主方案，`
`43家方案保存在支撑数据中，不与主表混用。"))，2)`

`section("5.4 问题四：稳态潜力与立即增产", [`
`text("在合同能力范围内，先将周产量设为可变，`
`以运输网络能够送达的产品当量为目标：")，`

`eq(r"C^*=\max_{y\ge 0}\sum_{i,j}\frac{(1-\ell_j)y_{ij}}{a_i},\quad`
`\sum_j y_{ij}\le u_i,\quad \sum_i y_{ij}\le 6000.")，`
`text(f"连续规划给出 $C^*=\{q4['steady\_capacity']:.3f\}$ 立方米/周，`
`对应稳态提升{q4['steady_increase_percent']:.3f}%。`

`供货能力总计{data['total_modeled_raw_supply_cap']:.3f}原料立方米/周，`
`低于8家总运输能力48000；当前主瓶颈是标准合同供给，而不是总运输容量。但为了减少损耗，`
`最佳几家转运商的6000上限仍会约束分配。")，`

`text(r"独立验证不重复调用线性规划。`
`将耗材系数较小的原料先分配给损耗较低的转运商，逐一填满容量。交换较优、`
`较劣两个配对带来的产品量增益为 $(1/a-1/b)(\ell_{\rm bad}-\ell_{\rm good})\ge 0$ ，`
`因此按转换效率与损耗排序的构造达到重排上界。本题允许拆分，且没有禁配边，`
`构造与线性规划一致。")，`

text("若将 C^* 直接用于第一周，两周目标库存同步增加，而初始库存仍为56400。
首周最多到货 C^* ，于是期末库存要求给出另一上界："),

eq(r"I_1=I_0+A_1-P\geq 2P,\quad A_1\leq C^*\quad\Longrightarrow\quad P\leq\frac{I_0+C^*}{3}."),

eq(r"P^*=\min\left(C^*,\frac{I_0+C^*}{3}\right)."),
text(f"代入得到 $P^*=\{q4['production_per_week']:.3f\}$ 立方米/周，
即从第一周起提高 $\{q4['immediate_increase_percent']:.3f\}\%$ 。"),

首周按最大产品当量采购到货，补足增加的储备；其后23周按 P^* 维持库存。

首周末库存达到 $\{q4['stock_target']:.3f\}$ 产品立方米当量，其后不下降。

该可行构造达到首周上界，完成‘上界—方案’闭环。"),

{"table": [{"计划阶段", "周数", "每周入库/产品立方米", "每周产量/产品立方米",
"期末库存/产品立方米"}]+ [{"label", str(num), f"{q4['arrivals_product_equivalent'] [t]:.3f}", f"{q4['production_per_week']:.3f}", f"{q4['ending_inventory_product_equivalent'] [t]:.3f}"} for label, num, t in (("首周", 1, 0), ("第2—24周", 23, 1))]},

text("若企业能在计划开始前把库存增加至两倍稳态产量，立即增产限制可以解除；
若初始库存低于假设值，结论向相反方向变化。题面未给初始库存，
故稳态数值与固定初始库存下的24周执行数值必须同时报告，不能把其中一个冒充无条件答案。"), 2)

section(" 六、模型检验与灵敏度分析", [])

section("6.1 预测误差与平均供货模型边界", [

text("按时间先后，使用第1—144周估计三个响应基线，在第145—192周比较，
随后对第193—240周作诊断。基线分别为数量比率、过原点最小二乘和逐周履约率中位数。
中位数在绝对误差上较小，但它估计典型供货而非平均供货；本模型涉及采购和到货总量核算，
因此采用数量比率，不把中位数冒充期望。"),

{"table": [{"响应基线", "145—192周有订货记录MAE", "每周总供货MAE"}]+ [{"volume_ratio": "总量比率", "least_squares": "过原点最小二乘", "median_ratio": "履约率中位数"} [r['method']], f"{r['validation_active_mae']:.3f}", f"{r['validation_total_weekly_mae']:.3f}"} for r in data['model_comparison']]},

text(f"数量比率在后48周有订货记录上的绝对误差为 $\{data['holdout'] ['active_mae']:.3f\}$ 原料立方米，
每周总量绝对误差为 $\{data['holdout'] ['weekly_total_mae']:.3f\}$ 立方米，
后者相对于实际周均总供货 $\{data['holdout'] ['actual_weekly_total_mean']:.3f\}$ 约为 $\{100*data['holdout'] ['weekly_total_mae']/data['holdout'] ['actual_weekly_total_mean']:.2f\}\%$ 。这一偏差不能支持‘预测准确、供货可靠’的强结论。"),

text("探索阶段已经查看过中位数基线在该后48周的汇总误差，
再依据估计对象不匹配修正为均值核算。因此这部分仅称按时间划分的诊断，
不再包装为完全未触碰的最终测试。更重要的是，验证使用的是历史实际订货条件，
不能把它当作对新增采购政策的反事实验证。未来连续供货能力仍需合同或新样本确认。"), 2)

section("6.2 数值独立性与硬约束复核", [

```

text("独立复算程序重新打开原始订货、供货与损耗工作簿，
不读取求解器内部约束矩阵来替代题意。它逐供应商核对供货上限和履约率，
逐周逐转运商汇总装车量，并以原料类别系数重新推算入库、库存和采购费用。共29项检查通过，
最大库存约束偏差仅为浮点舍入量。"),

text("最少家数由25家最佳损耗放松上界独立证明，稳态产能由重排构造独立证明；
两者不是同一个求解器换参数运行。混合整数经济阶段的最优性仍依赖HiGHS分支定界证书，
本文没有声称以另一程序穷举全部402家组合。基础三供应商合成算例另外用穷举核对最少家数，
并检查运输瓶颈下的连续构造。"),

text("附件A、B保存后重新读取全部指定区域，将每个非空数值与模型数组对应核验，
零订货或零转运保留空白；六个工作表的原表信息、合并区域和顺序均保留。
显示三位小数不改变底层数值，避免在大规模分配中积累手工抄录和过早舍入误差。"))],2)

section("6.3 能力假设与初始库存敏感性",[
figure("capacity-inventory","图3
标准合同供给能力变化对稳态和立即增产的影响；初始库存固定为56400产品立方米当量。"),
{"table": [{"供货能力倍数", "稳态产能/产品立方米每周", "立即产量/产品立方米
每周"}] + [{"r['contract_capacity_scale']:.1f}", f"{r['steady_capacity']:.
3f}", f"{r['immediate_capacity']:.3f}"] for r in
sensitivity[:5]}],

text("这组情景将所有标准合同供给同时缩放，直接影响可供原料的数量。
在0.8倍情景中连现有产能都不能满足，不能再讨论增加产量。
若转运损耗同时取各家历史非零周95%分位数，稳态产能重新求解；这个组合是较高损耗情景，
不是具有95%联合覆盖率的统计区间。"),

text(f"进一步用有订货周订货量90%分位作为合同上限，
会得到{next(r['steady_capacity'] for r in sensitivity if
'contract_envelope' in r):.3f}立方米/周的另一产能情景。
这与主模型的差别揭示了仅凭历史订单识别真实产能的困难，
不能因为更激进的假设给出更大数字就认为技术改造效果更好。"))],2)

section("6.4 供货风险与保护方案",[
figure("supply-stress","图4 问题二在供货量下降情景下的库存结余；
负值表示累计生产缺口，不表示允许出现物理负库存。"),

text("均值方案恰好补充每周消耗，持续少供会首先破坏两周储备，再耗尽库存。
供货降至模型值90%或80%时，安全目标从第一周便不再满足。供货高于模型值同样不能忽略：
企业仍需全部收购，原本满载的转运商可能超载，需要将多供部分改派有剩余能力的转运商。"),

text(f"为给出可实施的风险取舍，另设每家供货不低于均值模型90%、
各家损耗不超过其历史非零周95%分位的边界，按更不利的产品当量约束重新选合同。
得到{alt['Q2_bounded_response']['active_suppliers']}家供应商的保护方案，
其边界情景每周到货不少于{alt['Q2_bounded_response']['worst_case_weekly_arriv
als']:.3f}产品立方米，所有计划装车量仍满足6000上限。该方案的完整数量保存在支撑数据中。
"),

```

```
{ "table": [ [ "方案", "供应商数", "采购指数/万", "24周期末库存当量" ] ] + [ [ label, str(r['active_suppliers']), f"{r['purchase_cost_index']/10000:.4f}", f"{r['ending_inventory_product_equivalent'][-1]:.3f}" ] for label, r in
```

(("均值经济方案", q2), ("边界保护方案", alt['Q2_bounded_response']))] } ,

text("保护方案需要更多合同、更多采购及较大的平均库存，以换取声明扰动范围内的可行性；不把这个成本隐藏在‘鲁棒’二字后面。90%供货下界没有从合同或新观测得到保证，各转运商95%边界也没有联合校准，因此该方案不是95%可靠性保证，超出边界仍可能失败。")], 2)

section(" 七、模型评价与改进", [text("模型的优势是将三类原材料统一到产品当量，分别记录采购原料、损耗与库存；供应商最少数量和产能上限都有可检查的结构证据。字典序处理明确了采购、材料结构和损耗目标的优先级，成本临界条件揭示何时应该采用更偏A类的方案。完整结果由程序直接填写官方模板，便于逐周核验和调整。"),

text("主要不足来自供货信息而不是优化器精度：无订货周无法区分未被选择与无法供货，历史平均业务强度未证明能连续维持24周，量化诊断仍有明显误差。应优先向关键供应商确认周交付能力、停工周期和超供边界，再建立带可用性状态的滚动计划；不能靠提高求解器精度消除这些未知。"),

text("连续数量模型还没有纳入最小订单、整车批量和每家供应商的固定管理费。问题三310家、问题四首周402家活动供应商是这种简化的明显后果，实际执行应根据确认的费用比较43家备选和保护方案。题面运输与库存偏好也存在解释空间，若必须单家承运或初始库存不同，应相应增加整数分配约束和改写首周条件。"))]

section(" 八、结论", [text(f"第一问给出以历史边际保障量定义的重要性指标和50家完整名单。第二问在标准合同供给模型下最少26家，24周归一化采购成本{q2['purchase_cost_index']/10000:.4f}万，运输损耗{q2['expected_transport_loss']:.3f}立方米；数量下界与可行方案吻合。"),

text(f"第三问增加A、减少C后总原料量下降，但采购费用略增；需结合单位运输费、仓储费和管理费判断经济性。第四问稳态供给上限为{q4['steady_capacity']:.3f}立方米/周，在固定初始库存和首周储备目标下立即实施的恒定产量为{q4['production_per_week']:.3f}立方米/周。"),

text("结果随未来供货能力和初始状态改变。附件A、B给出主模型问题二至四的完整24周计划，支撑文件另给出43家材料偏好方案和48家边界保护方案，供企业在能力确认后选择。模型内最优和风险情景可行不等于真实生产验证。"))]

section("AI 工具使用声明", [text("本稿为赛前限时演练，由AI辅助完成题意解析、模型推导、编程、数值复核与中文写作，使用通用数值和排版工具。没有使用同题参赛论文或网络题解，模型预训练接触无法排除；本稿也不代表真实参赛队伍或人工专家已审核。"), text("使用详情：AI参与官方题面与四份附件的读取、边际保障指标构造、均值与中位数估计对象纠正、供应商混合整数规划、转运线性规划、首周库存上界推导、源数据复算、图表及结果模板生成。模型、代码与图表均由本次演练产出；未编造现场测量、合同承诺或人工签字。正式参赛时须由真实队伍核对适用年度规则，并提供实际使用记录。"))]

```

    section(" 参考文献",[
        text("[1] 全国大学生数学建模竞赛组委会.
2021年高教社杯全国大学生数学建模竞赛C题：生产企业原材料的订购与运输及附件1、附件2、
附件A、附件B。本次使用所提供官方原始文件。"),
        text("[2] SciPy developers. scipy.optimize.linprog 与
scipy.optimize.milp 软件接口。本次计算使用SciPy 1.16.1及其HiGHS后端；
具体求解设置和实现见源程序。"))
    appendix=run/"artifacts/submission-package/appendix.json"
    if
appendix.exists():sections.append(json.loads(appendix.read_text()))
    else:section(" 附录：结果文件与复算说明",[
        text("附件A、附件B保持原文件名，分别记录订货量和装车量。完整数值、候选方案、
约束核验及源程序随支撑包提供。当前正文生成时尚未接入代码附录，
必须在最终稿补齐后重新排版。"))],page=True)
    write(run/"paper/document.json",{ "title": "原材料订购与转运的分层优化模型"
,"sections":sections})
    print({"sections":len(sections),"full_code_appendix":appendix.exists
()},{flush=True})

if __name__=="__main__":
    ap=argparse.ArgumentParser();ap.add_argument("run",type=Path);compos
e(ap.parse_args().run)

prepare.py

"""Read frozen official data and evaluate order-to-supply baselines
chronologically."""
import argparse
import hashlib
import json
from pathlib import Path

import numpy as np
from openpyxl import load_workbook

def write(path, obj):
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(obj, ensure_ascii=False, indent=2,
allow_nan=False) + "\n", encoding="utf-8")

```

```

def rates(o, s, method):
    if method == "volume_ratio":
        return np.divide(s.sum(1), o.sum(1), out=np.zeros(len(o)),
where=o.sum(1)>0)
    if method == "least_squares":
        return np.divide((s*o).sum(1), (o*o).sum(1),
out=np.zeros(len(o)), where=(o*o).sum(1)>0)
    if method == "median_ratio":
        return np.array([np.median(b[a>0]/a[a>0]) if (a>0).any() else 0
for a,b in zip(o,s)])
    raise ValueError(method)

def read_data(run):
    p = next((run/"sources").glob("* 附件 1*"))
    w = load_workbook(p, read_only=True, data_only=True)
    rows = [list(s.iter_rows(min_row=2, max_row=403, values_only=True))
for s in w]
    w.close()
    ids=[r[0] for r in rows[0]]; kinds=[r[1] for r in rows[0]]
    assert len(ids)==402 and len(set(ids))==402
    assert [(r[0],r[1]) for r in rows[0]]==[(r[0],r[1]) for r in
rows[1]]
    o,s=[np.asarray([r[2:] for r in rr],float) for rr in rows]
    assert o.shape==s.shape==(402,240) and np.isfinite(o).all() and
np.isfinite(s).all()
    assert (o>=0).all() and (s>=0).all() and not ((o==0)&(s>0)).any()
    p2=next((run/"sources").glob("*附件2*"));w=load_workbook(p2,read_only
=True,data_only=True)
    rows=list(w.active.iter_rows(min_row=2,values_only=True));w.close()
    losses=np.asarray([r[1:] for r in rows],float)/100
    assert losses.shape==(8,240) and np.isfinite(losses).all() and
(losses>=0).all() and (losses<1).all()
    return ids,kinds,o,s,losses

def prepare(run):
    ids,kinds,o,s,losses=read_data(run)
    contract=json.loads((run/"problem_contract.json").read_text())
    contract["problem_interpretation"]="在供货不完全遵循订货、
转运存在损耗的条件下,评价402家供应商,制定24周订购与转运计划,并分析材料结构与增产。"

```

```

contract["official_outputs"]=["50家重要供应商名单", "问题2、3、
4未来24周订购和转运方案", "保持附件A、附件B原文件名和已有信息，仅填写规定区域"]
tasks=[("量化402家供应商供货特征并确定最重要的50家",["量化指标与模型",
"论文中50家供应商名单"]),
        ("确定满足生产需求至少需多少家供应商，
并给出最经济订购及最少损耗转运方案",["最少供应商数量", "24周订购与转运方案",
"实施效果分析"]),
        ("尽量多采购A、少采购C，并降低转运损耗",["新订购与转运方案",
"实施效果分析"]),
        ("根据现有供应商和转运商确定每周可提高的产能",["产能提升量",
"相应24周订购和转运方案"])]
contract["questions"]=[{"id":f"Q{i}", "source_locator":f"官方题面第1页问
题{i}及第2页附件说明",
"objective":task,
        "inputs":["附件1完整402×240订货与供货数据",
"附件2完整8×240损耗记录"], "outputs":outputs,
        "decision_variables":[] if i==1 else ["供应商订货量", "供应商—
转运商分配量"]+(["每周产能"] if i==4 else []),
        "hard_constraints":["每家转运商每周运输量不超过6000立方米",
"实际提供的原材料全部收购", "A/B/C每单位产品耗材0.6/0.66/0.72立方米"] if i>1
else [],
        "units":["原材料立方米", "产品立方米", "周",
"损耗率为百分数转换后的比例"],
        "ambiguities":["原题未给初始库存和绝对运储单价",
"尽可能两周库存与尽量单家转运为柔性表述；建模选择须另作说明"],
        "termination_condition":"完成该问的全部指定输出并验证"} for
i,(task,outputs) in enumerate(tasks,1)]
write(run/"problem_contract.json",contract)
a=np.array([{"A":.6,"B":.66,"C":.72}[k] for k in kinds])
price=np.array([{"A":1.2,"B":1.1,"C":1.}[k] for k in kinds])
methods=("volume_ratio","least_squares","median_ratio")
comparison=[]
for method in methods:
    r=rates(o[:, :144],s[:, :144],method);pred=r[:,None]*o[:,144:192]
    comparison.append({"method":method,"validation_active_mae":float
(np.abs(pred-s[:,144:192])[o[:,144:192]>0].mean()),
        "validation_total_weekly_mae":float(np.abs(pr
ed.sum(0)-s[:,144:192].sum(0)).mean())})
    # The planning estimand is expected volume, not conditional median
volume.
    # Preserve the MAE pilot as a diagnostic, never select a median as a
mean.

```

```

chosen="volume_ratio"

r_train=rates(o[:, :192], s[:, :192], chosen); pred=r_train[:, None]*o[:, 192:]
active=o[:, 192:]>0
test={"weeks":"193-240", "mean_accounting_method":chosen,
      "active_mae":float(np.abs(pred-s[:, 192:])[active].mean()),

"weekly_total_mae":float(np.abs(pred.sum(0)-s[:, 192:].sum(0)).mean()),
      "actual_weekly_total_mean":float(s[:, 192:].sum(0).mean()),
      "scope":"chronological diagnostic at observed historical
orders; median pilot exposed this block before the estimand correction,
so this is not an untouched final test or a counterfactual evaluation of
procurement"}
      fitted=rates(o, s, chosen)
      # Standard weekly contract capacity uses a historical active-week
mean.
      # No-order weeks are not evidence that the supplier was unavailable.
caps=np.array([row[row>0].mean() if (row>0).any() else 0 for row in
o])
caps90=np.array([np.quantile(row[row>0], .9) if (row>0).any() else 0
for row in o])
supply_caps=caps*fitted
carrier_mean=np.array([v[v>0].mean() for v in losses])
carrier_p95=np.array([np.quantile(v[v>0], .95) for v in losses])
equivalent=s/a[:, None]; total=equivalent.sum(0); demand=28200.
impact=np.minimum(demand, total)[None, :]-np.minimum(demand, total[None
, :]-equivalent)
score=impact.mean(1)
order=np.lexsort((np.arange(402), -score))
ranking=[]
for rank, i in enumerate(order, 1):
    mask=o[i]>0
    ranking.append({"rank":rank, "supplier":ids[i], "kind":kinds[i], "m
arginal_weekly_product_equivalent":float(score[i]),

"mean_supply":float(s[i].mean()), "ordered_weeks":int(mask.sum()),

"zero_supply_given_order":float((s[i, mask]==0).mean()) if mask.any()
else None,
                        "volume_fulfilment":float(s[i].sum()/o[i].sum())
if mask.any() else None,

```



```

        "order_cap":float(caps[i]),"fitted_supply_rate":
float(fitted[i]),"modeled_supply_cap":float(supply_caps[i])})
    basic={"ids":ids,"kinds":kinds,"conversion":a.tolist(),"price":price
.tolist(),"demand":demand,
        "order_cap":caps.tolist(),"order_cap_q90":caps90.tolist(),"su
pply_rate":fitted.tolist(),"supply_cap":supply_caps.tolist(),
        "carrier_mean_loss":carrier_mean.tolist(),"carrier_p95_loss":
carrier_p95.tolist(),"top50_indices":order[:50].tolist(),
        "initial_inventory_product_equivalent":2*demand,"weeks":24,
        "assumptions":["Initial inventory equals two weeks of planned
output; arrivals are usable in the same week; no lead time is
specified.",
            "Post-production stock target is two weeks.
Stationary expected plans keep this stock; realized risks are evaluated
separately.",
            "Expected supply is a nonnegative linear
response calibrated by volume ratio, within mean-positive-order contract
caps; repeating historical active-week activity is a modeling
assumption, not proven available capacity.",
            "Prices are indexed to C purchase price=1.
Actual transport/storage unit prices are absent; report physical volume
and stock rather than invented monetary totals.",
            "All modeled supplied raw material is
purchased and transported; a carrier can carry at most 6000 raw cubic
meters weekly.",
            "Split shipments are permitted by the soft
wording; number of splits will be reduced and reported.",
            "Min supplier count and capacity improvement
are conditional on fitted supply envelopes, not guaranteed future
physical capacities."]}
    stats={"suppliers":402,"weeks":240,"positive_orders":int((o>0).sum()
),"zero_supply_given_order_count":int(((o>0)&(s==0)).sum()),
        "nonzero_supply_without_order":int(((o==0)&(s>0)).sum()),"los
s_zero_missing_counts":(losses==0).sum(1).tolist(),
        "carrier_mean_loss":carrier_mean.tolist(),"carrier_p95_loss":
carrier_p95.tolist(),
        "train_weeks":"1-144","validation_weeks":"145-192","model_com
parison":comparison,"holdout":test,
        "total_modeled_raw_supply_cap":float(supply_caps.sum()),"tota
l_modeled_product_cap_before_transport":float((supply_caps/a).sum()),
        "top50_overlap_with_mean_supply_equivalent":int(len(set(order
[:50])&set(np.argsort(-equivalent.mean(1))[:50]))),

```

```

"source_sha256":{p.name:hashlib.sha256(p.read_bytes()).hexdigest() for p
in (run/"sources").iterdir() if p.is_file()}}
    write(run/"artifacts/facts.json",basic);write(run/"artifacts/data-an
alysis.json",stats);write(run/"artifacts/ranking.json",ranking)
    np.savez_compressed(run/"artifacts/observations.npz",orders=o,supply
=s,loss=losses)
    print(json.dumps(stats,ensure_ascii=False,indent=2))

```

```

if __name__=="__main__":
    ap=argparse.ArgumentParser();ap.add_argument("run",type=Path);prepar
e(ap.parse_args()).run)

```

renderer/package_support.py

```

"""Build a deterministic support ZIP and complete source appendix from
explicit files."""

```

```

import argparse
import hashlib
import json
import os
import shutil
import tempfile
import zipfile
from pathlib import Path, PurePosixPath

```

```

def _relative(value):
    if not isinstance(value, str) or not value or "\\\" in value or ":"
in value:
        raise ValueError("use a portable relative path")
    path = PurePosixPath(value)
    if path.is_absolute() or any(part in (".", "..") for part in
value.split("/")):
        raise ValueError("paths must remain inside their root")
    return path

```

```

def package_support(run, selection,
output="artifacts/submission-package"):
    run = Path(run).resolve()

```

```

        if (run / "state.json").is_file() and json.loads((run /
"state.json").read_text())["stage"] == "VERIFIED":
            raise ValueError("sealed runs are read-only; package a new
version")
        destination = run / _relative(output)
        destination.resolve().relative_to(run)
        if destination.exists():
            raise ValueError("package destination already exists; choose a
new version")
        if not isinstance(selection, dict) or not
isinstance(selection.get("files"), list) or not selection["files"]:
            raise ValueError("selection must list the files to include")
        entries = []
        names = set()
        for item in selection["files"]:
            if not isinstance(item, dict) or item.get("role") not in
("code", "data", "document"):
                raise ValueError("each file requires role
code/data/document")
            source = run / _relative(item.get("source"))
            source.resolve().relative_to(run)
            if any(parent.is_symlink() for parent in [source,
*source.parents] if parent != run and run in parent.parents):
                raise ValueError("select regular files, not symbolic links")
            if not source.is_file():
                raise ValueError("selected file does not exist")
            name = str(_relative(item.get("archive_path")))
            if name.casefold() in names or name.casefold() in
{"manifest.json", "appendix.json"}:
                raise ValueError("duplicate or reserved archive filename")
            names.add(name.casefold())
            if source.stat().st_size > 50_000_000:
                raise ValueError("selected file exceeds packager memory
limit (not an official rule)")
            content = source.read_bytes()
            code = content.decode("utf-8-sig") if item["role"] == "code"
else None
            entries.append({"path": name, "role": item["role"], "bytes":
len(content),
                            "sha256": hashlib.sha256(content).hexdigest(),
"content": content, "code": code})
            entries.sort(key=lambda entry: entry["path"])

```

```

    manifest = {"schema_version": "1.0", "scope": "explicitly selected
support files; completeness requires task review",
               "files": [{key: row[key] for key in ("path", "role",
"bytes", "sha256")}] for row in entries]}
    blocks = [{"text": "以下文件与支撑包同源生成。完整代码逐文件列出。
此清单不表示人工核验或正式提交已经完成。"},
              {"table": [{"文件", "用途类别", "字节数"}] + [[row["path"],
row["role"], row["bytes"]] for row in entries] + [{"manifest.json",
"文件校验清单", "随包生成"}], "long_table": True]}
    blocks.extend({"code": row["code"], "filename": row["path"],
"source_sha256": row["sha256"]}
                  for row in entries if row["role"] == "code")
    appendix = {"id": "support-appendix", "title": "附录：
支撑文件与完整源程序", "page_break_before": True, "blocks": blocks}
    destination.parent.mkdir(parents=True, exist_ok=True)
    temporary = Path(tempfile.mkdtemp(prefix=".package-",
dir=destination.parent))
    try:
        archive_path = temporary / "support.zip"
        with zipfile.ZipFile(archive_path, "w",
compression=zipfile.ZIP_DEFLATED) as archive:
            def add(name, content):
                info = zipfile.ZipInfo(name, date_time=(2000,1,1,0,0,0))
                info.compress_type = zipfile.ZIP_DEFLATED
                info.create_system = 3
                info.external_attr = 0o100644 << 16
                archive.writestr(info, content)
            for row in entries:
                add(row["path"], row["content"])
            add("manifest.json", (json.dumps(manifest,
ensure_ascii=False, indent=2)+"\n").encode())
            if archive_path.stat().st_size > 20_000_000:
                raise ValueError("support ZIP exceeds conservative 20 MB
limit")
        appendix_path = temporary / "appendix.json"
        appendix_path.write_text(json.dumps(appendix,
ensure_ascii=False, indent=2)+"\n", encoding="utf-8")
        receipt = {"archive": "support.zip", "sha256":
hashlib.sha256(archive_path.read_bytes()).hexdigest(),
                  "md5": hashlib.md5(archive_path.read_bytes(),
usedforsecurity=False).hexdigest(),

```

```

        "appendix_sha256":
hashlib.sha256(appendix_path.read_bytes()).hexdigest(),
        "file_count": len(entries), "code_count":
sum(row["role"] == "code" for row in entries),
        "submission_ready": False, "manual_review":
"PENDING", "manifest": manifest}
    (temporary / "package_receipt.json").write_text(json.dumps(recei
pt,ensure_ascii=False,indent=2)+"\n",encoding="utf-8")
    os.rename(temporary,destination)
except BaseException:
    shutil.rmtree(temporary)
    raise
return receipt

```

```

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("run",type=Path)
    parser.add_argument("--selection",type=Path,required=True)

    parser.add_argument("--output",default="artifacts/submission-package")
    args = parser.parse_args()
    result=package_support(args.run,json.loads(args.selection.read_text(
encoding="utf-8")),args.output)
    print(json.dumps({key:value for key,value in result.items() if
key!="manifest"},ensure_ascii=False,indent=2))

```

renderer/render_latex.py

```

"""Typeset Chinese papers with real inline/display mathematics using
XeTeX/Tectonic."""
import argparse
import hashlib
import json
import os
import platform
import re
import shutil
import subprocess
from pathlib import Path

```

```

def escape_text(text):
    replacements = {"\\": r"\textbackslash{}", "&": r"\&", "%": r"\%",
"$": r"\$", "#": r"\#",
        "_": r"\_", "{": r"\{", "(": r"\(", "~":
r"\textasciitilde{}", "^": r"\textasciicircum{}"}
    return "".join(replacements.get(character, character) for character
in str(text))

def inline(text):
    text = str(text)
    if text.count("$") % 2:
        raise ValueError("unbalanced inline math delimiters")
    return "".join(piece if piece.startswith("$") and
piece.endswith("$") else plain_with_units(piece)
        for piece in re.split(r"(\^[^$]*\$)", text))

def plain_with_units(text):
    units = {"cm · min-1": r"$\mathrm{cm}\backslash,\backslash\mathrm{min}^{\{-1\}}$", "° · s":
r"${}^{\circ}\backslash\mathrm{C}\backslash!\cdot!\backslash\mathrm{s}$",
        "°/s": r"${}^{\circ}\backslash\mathrm{C}/\backslash\mathrm{s}$", "°":
r"${}^{\circ}\backslash\mathrm{C}$"}
    pattern = "(" + "|".join(re.escape(unit) for unit in units) + ")"
    return "".join(units.get(piece, escape_text(piece)) for piece in
re.split(pattern, text))

def render_latex(run, source, *, compiler=None, cache_dir=None,
compile_pdf=True):
    run = Path(run).resolve()
    source = Path(source).resolve()
    if (run / "state.json").exists() and json.loads((run /
"state.json").read_text())["stage"] == "VERIFIED":
        raise ValueError("create a new version before typesetting a
sealed run")
    data = json.loads(source.read_text(encoding="utf-8"))
    output = run / "paper"
    output.mkdir(exist_ok=True)
    document_sha256 = hashlib.sha256(source.read_bytes()).hexdigest()

```

```

(output /
"render_report.json").write_text(json.dumps({"render_status": "PENDING",
"source": source.name,

"document_sha256": document_sha256,

"note": "An
existing PDF may be from an earlier generation; not accepted until
compilation succeeds."},

ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    if platform.system() == "Darwin":
        fonts = r"\setmainfont{Times New Roman}\setCJKmainfont{Songti
SC}\setCJKsansfont{Heiti SC}\setCJKmonofont{Songti SC}"
    else:
        fonts = r"\setmainfont{TeX Gyre Termes}\setCJKmainfont{FandolSon
g-Regular.otf}[BoldFont=FandolSong-Bold.otf]\setCJKsansfont{FandolHei-Re
gular.otf}\setCJKmonofont{FandolSong-Regular.otf}"
        has_code = any("code" in block for section in data["sections"] for
block in section["blocks"])
        if has_code:
            fonts += r"\usepackage{fvextra}"
            if platform.system() == "Darwin":
                fonts += r"\setmonofont{Menlo}"
            else:
                fonts += r"\IfFontExistsTF{DejaVu Sans
Mono}{\setmonofont{DejaVu Sans Mono}}{}"
                # Celsius is present in Chinese fonts but absent from
Menlo/DejaVu Sans Mono.
                # Preserve the literal source character in code instead of
replacing it.
                fonts += r'\xeCJKDeclareCharClass{CJK}{\2103}'
            code_receipts = []
            preamble = r"""\documentclass[12pt,a4paper]{article}
\usepackage{fontspec,xecjk,amsmath,amssymb,geometry,graphicx,booktabs,ta
bularx,array}
\usepackage{titlesec,caption,placeins,indentfirst,float,longtable}
\geometry{margin=28mm,footskip=12mm}
FONTS
\XeTeXlinebreaklocale "zh"
\XeTeXlinebreakskip=0pt plus 1pt
\setlength{\parindent}{2em}
\setlength{\parskip}{0.25em}

```

```

\linespread{1.35}
\titleformat{\section}{\sffamily\bfseries\large}{}{0em}{}
\titleformat{\subsection}{\sffamily\bfseries\normalsize}{}{0em}{}
\titleformat{\subsubsection}{\sffamily\normalsize}{}{0em}{}
\titlespacing*{\section}{0pt}{1.2em}{0.7em}
\captionsetup{font=small,skip=6pt}
\renewcommand{\arraystretch}{1.35}
\setlength{\emergencystretch}{2em}
\begin{document}
""".replace("FONTS", fonts)
    lines = [preamble, r"\begin{center}{\sffamily\bfseries\LARGE " +
inline(data["title"]) + r"}\end{center}"]
    markdown = ["# " + data["title"], ""]
    if data.get("status_note"):
        lines.append(r"\begin{center}\small " +
inline(data["status_note"]) + r"\end{center}")
        markdown.extend(["> " + data["status_note"], ""])
    for section in data["sections"]:
        lines.append(r"\FloatBarrier")
        if section.get("page_break_before"):
            lines.append(r"\clearpage")
        level = section.get("level", 1)
        if level not in (1, 2, 3):
            raise ValueError("section level must be 1, 2 or 3")
        command = {1: "section", 2: "subsection", 3:
"subsubsection"}[level]
        lines.append("\\ " + command + "*{" + inline(section["title"]) +
"}")
        markdown.extend(["#" * (level + 1) + " " + section["title"],
""])
        for block in section["blocks"]:
            if "text" in block:
                lead = block.get("lead", "")
                prefix = r"{\sffamily\bfseries " + inline(lead) + "}" if
lead else ""
                lines.extend([prefix + inline(block["text"]), r"\par"])
                markdown.extend(["**" + lead + "** " if lead else ""] +
block["text"], ""])
            elif "equation" in block:
                formula = block["equation"]
                lines.extend([r"\begin{equation}", formula,
r"\end{equation}"])

```



```

        markdown.extend(["$$", formula, "$$", ""])
    elif "code" in block:
        code = block["code"]
        if not isinstance(code, str) or not code.strip():
            raise ValueError("source-code blocks require
nonempty text")
        name = block.get("filename", "source")
        code_file = output /
f"source-{len(code_receipts)+1:03d}.txt"
        code_file.write_text(code, encoding="utf-8")
        code_receipts.append({"filename": name, "utf8_sha256":
hashlib.sha256(code.encode("utf-8")).hexdigest()})
        lines.extend([r"\subsubsection*{" + escape_text(name) +
"},

r"\VerbatimInput[fontsize=\footnotesize,breaklines=true,breakanywhere=tr
ue,breakautoindent=false,breakindent=0pt,breaksymbolleft={},breaksymbolr
ight={},breakanywheresymbolpre={},breakanywheresymbolpost={}]{" +
code_file.name + "}")

        longest = max((len(match.group()) for match in
re.finditer(r"`+", code)), default=0)
        fence = "`" * max(3, longest+1)
        markdown.extend(["### " + name, "", fence,
code.rstrip("\n"), fence, ""])
    elif "table" in block:
        rows = block["table"]
        columns = len(rows[0])
        if any(len(row) != columns for row in rows):
            raise ValueError("ragged paper table")
        if block.get("caption"):
            markdown.extend([block["caption"], ""])
        long_table = block.get("long_table", False)
        if long_table:
            width = r"\dimexpr\linewidth-" + str(2*columns) +
r"\tabcolsep)/" + str(columns) + r"\relax"
            lines.extend([r"\begin{group}\small\linespread{1.1}\se
lectfont\renewcommand{\arraystretch}{1.15}",
r"\begin{longtable}{" +
(r">\raggedright\arraybackslash}p{" + width + "}")*columns + "}")])
        if block.get("caption"):
            lines.append(r"\caption*{" +
inline(block["caption"]) + r"}\\")

```

```

        lines.append(r"\toprule")
    else:
        lines.append(r"\begin{table}[H]\centering\small")
        if block.get("caption"):
            lines.append(r"\caption*{" +
inline(block["caption"]) + "}")
            lines.extend([r"\begin{tabularx}{\linewidth}{ " +
r">\raggedright\arraybackslash}X" * columns + "}", r"\toprule"])
            for index, row in enumerate(rows):
                formatted = []
                for cell in row:
                    if index == 0 and "/" in str(cell) and "$" not
in str(cell):
                        label, unit = str(cell).split("/", 1)
                        formatted.append(r"\shortstack[l]{ " +
inline(label) + r"\\(" + inline(unit) + ")}")
                    else:
                        formatted.append(inline(cell))
                lines.append(" & ".join(formatted) + r" \\\")
                if index == 0:
                    lines.append(r"\midrule")
                    if long_table:
                        lines.extend([r"\endfirsthead\toprule", " &
".join(formatted) + r" \\\",

r"\midrule\endhead\bottomrule\endfoot"])
                    markdown.append("| " + " | ".join(str(cell) for cell
in row) + " |")
                if index == 0:
                    markdown.append("| " + " | ".join("----" for _ in
row) + " |")
            if long_table:
                lines.append(r"\end{longtable}\endgroup")
            else:
                lines.append(r"\bottomrule\end{tabularx}\end{table}")
                markdown.append("")
    elif "image" in block:
        image = (run / block["image"]).resolve()
        image.relative_to(run)
        relative = os.path.relpath(image,
output).replace(os.sep, "/")

```

```

        if any(character in relative for character in
"{}%\\n\\r"):
            raise ValueError("unsupported characters in figure
path")

        lines.extend([r"\begin{figure}[H]\centering",

r"\includegraphics[width=\linewidth]{\detokenize{" + relative + "}}"])
        if block.get("caption"):
            lines.append(r"\caption*{" +
inline(block["caption"]) + "}")
            lines.append(r"\end{figure}")
            markdown.extend([f"! [{block.get('caption',
'')}]({relative})", ""])
        else:
            raise ValueError("unknown paper block")
        lines.append(r"\end{document}")
        tex = output / "main.tex"
        tex.write_text("\n".join(lines) + "\n", encoding="utf-8")
        (output / "main.md").write_text("\n".join(markdown),
encoding="utf-8")
        if compile_pdf:
            executable = compiler or shutil.which("tectonic") or
shutil.which("xelatex")
            if not executable:
                raise ValueError("TeX source written; install
Tectonic/XeLaTeX or supply --compiler. Do not fall back to plaintext
formulas.")
            env = os.environ.copy()
            env["TECTONIC_CACHE_DIR"] = str(Path(cache_dir).resolve() if
cache_dir else run / "tmp/tectonic-cache")
            if "tectonic" in Path(executable).name.lower():
                command = [str(executable), "-X", "compile", "--untrusted",
"--keep-logs", "--outdir", str(output), str(tex)]
            else:
                command = [str(executable), "-no-shell-escape",
"-interaction=nonstopmode", "-halt-on-error", "-output-directory",
str(output), str(tex)]
            completed = subprocess.run(command, cwd=output, env=env,
capture_output=True, text=True)
            (output / "typesetting.log").write_text(completed.stdout +
completed.stderr, encoding="utf-8")
            if completed.returncode:

```

```

        raise RuntimeError("TeX compilation failed; inspect
paper/typesetting.log")
        log = (output / "main.log").read_text(encoding="utf-8",
errors="replace") if (output / "main.log").is_file() else
completed.stdout + completed.stderr
        if "Missing character:" in log:
            raise RuntimeError("TeX reported missing glyphs; inspect
paper/main.log")
        from pypdf import PdfReader
        pdf = PdfReader(output / "main.pdf")
        font_names = sorted({str(font.get_object().get("/BaseFont", ""))
for page in pdf.pages
                                for font in page["/Resources"].get("/Font",
{ }).values())})
        report = {"pages": len(pdf.pages), "encrypted":
pdf.is_encrypted,
                "text_characters": sum(len(page.extract_text() or "")
for page in pdf.pages),
                "visual_review": "PENDING", "source": source.name,
"backend": "latex",
                "fonts": font_names, "pdf_sha256":
hashlib.sha256((output / "main.pdf").read_bytes()).hexdigest(),
                "document_sha256": document_sha256, "render_status":
"COMPILED", "code_blocks": code_receipts}
        (output / "render_report.json").write_text(json.dumps(report,
ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
        print({"tex": str(tex), "pdf_compiled": compile_pdf,
"visual_review": "PENDING"})

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--run", type=Path, required=True)
    parser.add_argument("--source", type=Path, required=True)
    parser.add_argument("--compiler")
    parser.add_argument("--cache-dir", type=Path)
    parser.add_argument("--source-only", action="store_true")
    args = parser.parse_args()
    render_latex(args.run, args.source, compiler=args.compiler,
cache_dir=args.cache_dir, compile_pdf=not args.source_only)

```

renderer/render_paper.py

```
"""Render a structured paper to matching Markdown and A4 PDF. Optional
dependencies."""
import argparse
import html
import json
from pathlib import Path

def render(run: Path, source: Path, font: Path | None = None, *,
          latex_compiler=None, cache_dir=None):
    data = json.loads(source.read_text(encoding="utf-8"))
    has_math = any("equation" in block or "code" in block or "$" in
                  str(block.get("text", "")) or "$" in str(block.get("table", ""))
                  for section in data["sections"] for block in
                  section["blocks"])
    if has_math:
        from render_latex import render_latex
        return render_latex(run, source, compiler=latex_compiler,
                           cache_dir=cache_dir)
    from reportlab.lib import colors
    from reportlab.lib.enums import TA_CENTER
    from reportlab.lib.styles import getSampleStyleSheet, ParagraphStyle
    from reportlab.lib.units import cm
    from reportlab.pdfbase import pdfmetrics
    from reportlab.pdfbase.ttfonts import TTFont
    from reportlab.platypus import SimpleDocTemplate, Paragraph, Spacer,
    Table, TableStyle, Image, PageBreak, KeepTogether
    from pypdf import PdfReader
    run = run.resolve()
    if (run / "state.json").is_file() and json.loads((run /
    "state.json").read_text())["stage"] == "VERIFIED":
        raise ValueError("create a new version before rendering a sealed
run")
    candidates = [font] if font else [
        Path("/System/Library/Fonts/Supplemental/Songti.ttc"),
        Path("/usr/share/fonts/truetype/arphic/uming.ttc"),
    ]
    font_path = next((path for path in candidates if path and
    path.is_file()), None)
    if font_path is None:
```

```

        raise ValueError("provide an embeddable CJK TrueType font using
--font")
    paper_font = TTFont("PaperFont", str(font_path), subfontIndex=6 if
font_path.name == "Songti.ttc" else 0)
    pdfmetrics.registerFont(paper_font)
    content = json.dumps(data, ensure_ascii=False)
    missing = sorted({character for character in content if
ord(character) > 127 and ord(character) not in
paper_font.face.charToGlyph})
    if missing:
        raise ValueError("selected font lacks glyphs: " +
"".join(missing))
    styles = getSampleStyleSheet()
    body = ParagraphStyle("CJKBody", fontName="PaperFont",
fontSize=10.5, leading=17, wordWrap="CJK", spaceAfter=7)
    title = ParagraphStyle("CJKTitle", parent=body, fontSize=19,
leading=26, alignment=TA_CENTER, spaceAfter=16)
    heading = ParagraphStyle("CJKHeading", parent=body, fontSize=13,
leading=20, spaceBefore=13, spaceAfter=8, keepWithNext=True)
    caption = ParagraphStyle("CJKCaption", parent=body, fontSize=9,
leading=13, alignment=TA_CENTER, textColor=colors.HexColor("#475569"))
    cell_style = ParagraphStyle("Cell", parent=body, fontSize=8.5,
leading=12, spaceAfter=0)
    story = [Paragraph(html.escape(data["title"]), title)]
    markdown = ["# " + data["title"], ""]
    if data.get("status_note"):
        story.append(Paragraph(html.escape(data["status_note"]),
caption))
    story.append(Spacer(1, .25 * cm))
    markdown.extend(["> " + data["status_note"], ""])
    for section in data["sections"]:
        if section.get("page_break_before"):
            story.append(PageBreak())
        story.append(Paragraph(html.escape(section["title"]), heading))
        markdown.extend(["## " + section["title"], ""])
        for block in section["blocks"]:
            if "text" in block:
                text = block["text"]
                story.append(Paragraph(html.escape(text), body))
                markdown.extend([text, ""])
            elif "table" in block:
                rows = block["table"]

```

```

        width = 16 * cm / len(rows[0])
        table = Table([[Paragraph(html.escape(str(cell)),
cell_style) for cell in row] for row in rows],
                      colWidths=[width] * len(rows[0]),
repeatRows=1, hAlign="CENTER")
        table.setStyle(TableStyle([
            ("BACKGROUND", (0, 0), (-1, 0),
colors.HexColor("#e8eef2")),
            ("LINEABOVE", (0, 0), (-1, 0), .8,
colors.HexColor("#334155")),
            ("LINEBELOW", (0, 0), (-1, 0), .5,
colors.HexColor("#334155")),
            ("LINEBELOW", (0, -1), (-1, -1), .8,
colors.HexColor("#334155")),
            ("VALIGN", (0, 0), (-1, -1), "TOP"),
            ("TOPPADDING", (0, 0), (-1, -1), 6),
            ("BOTTOMPADDING", (0, 0), (-1, -1), 6),
        ]))
        story.append(KeepTogether([table, Spacer(1, .2 * cm)])
if len(rows) <= 10 else table)
        for index, row in enumerate(rows):
            markdown.append("| " + " |
".join(str(cell).replace("|", "\\|") for cell in row) + " |")
            if index == 0:
                markdown.append("| " + " | ".join("----" for _ in
row) + " |")
            markdown.append("")
        elif "image" in block:
            path = (run / block["image"]).resolve()
            path.relative_to(run)
            graphic = Image(str(path))
            width = min(float(block.get("width_cm", 16)), 16) * cm
            graphic.drawHeight *= width / graphic.drawWidth
            graphic.drawWidth = width
            image_block = [graphic]
            if block.get("caption"):
                image_block.append(Paragraph(html.escape(block["caption"]), caption))
            story.append(KeepTogether(image_block))
            markdown.extend([f"![{block.get('caption',
'')}]({../{block['image']})", ""])
            output = run / "paper"

```

```

    output.mkdir(exist_ok=True)
    (output / "main.md").write_text("\n".join(markdown),
encoding="utf-8")
    def footer(canvas, document):
        canvas.saveState()
        canvas.setFont("PaperFont", 9)
        canvas.setFillColor(colors.HexColor("#64748b"))
        canvas.drawCentredString(10.5 * cm, 1.2 * cm,
str(document.page))
        canvas.restoreState()
    document = SimpleDocTemplate(str(output / "main.pdf"), pagesize=(21
* cm, 29.7 * cm),
                                leftMargin=2.5 * cm, rightMargin=2.5 *
cm, topMargin=2.5 * cm, bottomMargin=2.5 * cm,
                                title=data["title"], author="")
    document.build(story, onFirstPage=footer, onLaterPages=footer)
    pdf = PdfReader(output / "main.pdf")
    report = {"pages": len(pdf.pages), "encrypted": pdf.is_encrypted,
            "text_characters": sum(len(page.extract_text() or "") for
page in pdf.pages),
            "visual_review": "PENDING", "source": source.name,
"embedded_font": font_path.name}
    (output / "render_report.json").write_text(json.dumps(report,
ensure_ascii=False, indent=2) + "\n", encoding="utf-8")
    print(report)

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--run", type=Path, required=True)
    parser.add_argument("--source", type=Path, required=True)
    parser.add_argument("--font", type=Path)
    parser.add_argument("--latex-compiler")
    parser.add_argument("--cache-dir", type=Path)
    args = parser.parse_args()
    render(args.run, args.source, args.font,
latex_compiler=args.latex_compiler, cache_dir=args.cache_dir)

```



```

run_all.py

"""Reproduce the 2021 C calculations; optionally rebuild official
workbooks and PDF."""
import argparse
import json
import shutil
import subprocess
import sys
from pathlib import Path

HERE=Path(__file__).resolve().parent

def run(args):
    out=args.output.resolve();out.mkdir(parents=True,exist_ok=False)
    (out/"sources").mkdir();(out/"artifacts").mkdir()
    sources=sorted(args.source.glob("*.xlsx"))+[args.source/"CUMCM2021-C
.pdf"]
    if len(sources)!=5 or not all(p.is_file() for p in sources):raise
ValueError("official PDF and four original workbooks required")
    for p in sources:shutil.copy2(p,out/"sources"/p.name)
    (out/"problem_contract.json").write_text(json.dumps({"schema_version
":"1.0","run_id":"supply-2021c-reproduction","title":"2021 C
原材料订购与运输"},ensure_ascii=False),encoding="utf-8")
    for script in
("prepare.py","solve.py","verify.py","alternatives.py"):
subprocess.run([sys.executable,str(HERE/script),str(out)],check=True)
    if args.deliver:
        bundled=Path.home()/"cache/codex-runtimes/codex-primary-runtime
/dependencies/node"
        node=args.node or bundled/"bin/node"
        modules=args.artifact_modules or bundled/"node_modules"
        if not node.is_file() or not
(modules/"@oai/artifact-tool").is_dir():
            raise ValueError("workbook export requires available Codex
artifact-tool runtime; provide --node and --artifact-modules")
        link=HERE/"node_modules"
        if not link.exists():link.symlink_to(modules.resolve(),target_is
_directory=True)

```

```

        subprocess.run([str(node), str(HERE/"workbooks.mjs"), str(out), "fi
ll"], check=True)
        for script in
("check_workbooks.py", "figures.py", "package.py", "paper.py"):

subprocess.run([sys.executable, str(HERE/script), str(out)], check=True)
        renderer=HERE/"renderer/render_paper.py"
        if not renderer.exists(): renderer=HERE.parents[1]/"skill/cumcm-r
eliable-paper/scripts/render_paper.py"
        cmd=[sys.executable, str(renderer), "--run", str(out), "--source", st
r(out/"paper/document.json")]
        if args.latex_compiler: cmd.extend(["--latex-compiler", str(args.l
atex_compiler)])
        if
args.cache_dir: cmd.extend(["--cache-dir", str(args.cache_dir)])
        subprocess.run(cmd, check=True)
        print("Reproduction finished. Original timed-trial acceptance is not
changed.")

```

```

if __name__=="__main__":
    ap=argparse.ArgumentParser(description=__doc__)
    ap.add_argument("--source", type=Path, required=True); ap.add_argument(
"--output", type=Path, required=True)
    ap.add_argument("--deliver", action="store_true"); ap.add_argument("--
node", type=Path); ap.add_argument("--artifact-modules", type=Path)
    ap.add_argument("--latex-compiler", type=Path); ap.add_argument("--cac
he-dir", type=Path)
    run(ap.parse_args())

```

solve.py

```

"""Mixed-integer supplier choice and lexicographic material/transport
planning."""

```

```

import argparse
import json
from pathlib import Path

```

```

import numpy as np
from scipy.optimize import Bounds, LinearConstraint, linprog, milp
from scipy.sparse import coo_matrix, csr_matrix, hstack, vstack
from prepare import write

```

```

def network(facts, scale=1., cap_override=None, losses=None):
    cap=np.asarray(facts["supply_cap"] if cap_override is None else
cap_override)*scale
    a=np.array(facts["conversion"]);price=np.array(facts["price"])
    loss=np.array(facts["carrier_mean_loss"] if losses is None else
losses)
    n=len(cap);m=8*n
    rows=np.r_[np.repeat(np.arange(n),8), n+np.tile(np.arange(8),n)]
    cols=np.r_[np.arange(m),np.arange(m)]
    A=coo_matrix((np.ones(2*m),(rows,cols)),shape=(n+8,m)).tocsr()
    b=np.r_[cap,np.full(8,6000.)]
    yield_=((1-loss)[None,:]/a[:,None]).ravel()
    cost=np.repeat(price,8);rawloss=np.tile(loss,n)
    return cap,a,price,loss,A,b,yield_,cost,rawloss

def lp_solve(c,A,b,Aeq=None,beq=None):
    r=linprog(c,A_ub=A,b_ub=b,A_eq=Aeq,b_eq=beq,bounds=(0,None),method="
highs")
    if not r.success:raise ValueError(r.message)
    # Independently checkable primal and dual objective values for this
LP.
    dual=float(np.dot(b,r.ineqlin.marginals))
    if beq is not None:dual+=float(np.dot(beq,r.eqlin.marginals))
    return r,{"objective":float(r.fun),"dual_objective":dual,"gap":float
(r.fun-dual),"status":r.message}

def count_suppliers(facts, target, allowed=None):
    cap,a,price,loss,A,b,yield_,cost,rawloss=network(facts)
    n=len(cap);m=8*n
    if allowed is not None:
        mask=np.zeros(n);mask[allowed]=1;cap=cap*mask
    # Supply bounds bind binary contract activation; carrier bounds
remain raw m3.
    choice=coo_matrix((-cap,(np.arange(n),np.arange(n))),shape=(n+8,n)).
tocsr()

rows=hstack([A,choice]).tocsr();rhs=np.r_[np.zeros(n),np.full(8,6000.)]

```

```

        rows=vstack([rows,csr_matrix(np.r_[yield_,np.zeros(n)])[None,:])]).tocsr();rhs=np.r_[rhs,-target]
        c=np.r_[np.zeros(m),np.ones(n)]
        bounds=Bounds(np.zeros(m+n),np.r_[np.full(m,np.inf),np.ones(n)])
        integer=np.r_[np.zeros(m),np.ones(n)]
        r=milp(c,integrality=integer,bounds=bounds,constraints=LinearConstraint(rows,-np.inf,rhs),options={"time_limit":90.,"mip_rel_gap":0.})
        if r.x is None:raise ValueError(r.message)
        number=int(np rint(r.x[m:]).sum())
        optimistic=np.sort(cap/np.array(facts["conversion"])*(1-min(loss)))[::-1]

```

```

optimistic_count=int(np.searchsorted(np.cumsum(optimistic),target)+1)
    if not r.success and optimistic_count!=number:raise
ValueError("minimum supplier count not certified before time cap")
    # Economic optimum among all sets of that minimum size.
    rows2=vstack([rows,csr_matrix(np.r_[np.zeros(m),np.ones(n)])[None,:])]).tocsr();rhs2=np.r_[rhs,number]

```

```

economic=milp(np.r_[cost,np.zeros(n)],integrality=integer,bounds=bounds,
               constraints=LinearConstraint(rows2,-np.inf,rhs2),options={"time_limit":90.,"mip_rel_gap":1e-7})
    if economic.x is None:raise ValueError(economic.message)
    x=economic.x[:m].reshape(n,8)
    certificate={"minimum_supplier_count":number,"optimistic_sorted_capacity_lower_bound":optimistic_count,
                "count_solver_status":r.message,"count_mip_dual_bound":float(r.mip_dual_bound),
                "economic_status":economic.message,"economic_mip_gap":float(economic.mip_gap),
                "economic_lower_bound":float(economic.mip_dual_bound),"economic_objective":float(economic.fun)}
    return x,certificate

```

```

def material_plan(facts,target,mode="cost",cap_override=None):
    cap,a,price,loss,A,b,yield_,cost,rawloss=network(facts,cap_override=cap_override)
    eq=csr_matrix(yield_[None,:]);beq=np.array([target]);proof=[]
    objectives=[]
    if mode=="A_then_C":
        isA=np.repeat(np.array(facts["kinds"]),"A",8)

```

```

        isC=np.repeat(np.array(facts["kinds"]),"C",8)
        objectives=[-yield_*isA,isC.astype(float)]
    objectives.append(cost)
    for objective in objectives:
        r,cert=lp_solve(objective,A,b,eq,beq);proof.append(cert)
        # Exact equality retains the earlier lexicographic optimum.
        eq=vstack([eq,csr_matrix(objective[None,:])]).tocsr();beq=np.r_[
beq,r.fun]
        r,cert=lp_solve(rawloss,A,b,eq,beq);proof.append(cert)
    return r.x.reshape(len(cap),8),proof

def transfer_fixed_supply(facts,x,target):
    cap,a,price,loss,A,b,yield_,cost,rawloss=network(facts)
    n=len(cap);supply=x.sum(1)
    eq=A[:n];rhs=supply
    rows=vstack([A[n:],csr_matrix(-yield_[None,:])]).tocsr();ub=np.r_[b[
n:],-target]
    r,proof=lp_solve(rawloss,rows,ub,eq,rhs)
    return r.x.reshape(n,8),proof

def capacity(facts,scale=1.,losses=None,cap_override=None):
    cap,a,price,loss,A,b,yield_,cost,rawloss=network(facts,scale,cap_ove
rride,losses)
    r,proof=lp_solve(-yield_,A,b)
    return -r.fun,r.x.reshape(len(cap),8),proof

def record_plan(facts,shipments,production):
    y=np.asarray(shipments);a=np.array(facts["conversion"]);r=np.array(f
acts["supply_rate"])

    loss=np.array(facts["carrier_mean_loss"]);price=np.array(facts["price"])
    expected_supply=y.sum(2)
    orders=np.divide(expected_supply,r[None,:],out=np.zeros_like(expecte
d_supply),where=r[None,:]>0)
    arrivals=(y*(1-loss)[None,None,:]/a[None,:,None]).sum((1,2))
    inventory=facts["initial_inventory_product_equivalent"]+np.cumsum(ar
rivals-production)

```

```

        raw_by_type={k:float(expected_supply[:,np.array(facts["kinds"])==k].
sum()) for k in
"ABC"}
        nlinks=(y>1e-7).sum(2)
        return {"production_per_week":float(production),"orders":orders.tolist(),
"shipments":y.tolist(),
                "arrivals_product_equivalent":arrivals.tolist(),"ending_inventory_product_equivalent":inventory.tolist(),
                "purchase_cost_index":float((expected_supply*price).sum()),"
raw_material_volume":float(expected_supply.sum()),
                "raw_by_type":raw_by_type,"expected_transport_loss":float((y
*loss[None,None,:]).sum()),
                "active_suppliers":int((expected_supply.sum(0)>1e-6).sum()),
                "split_supplier_weeks":int((nlinks>1).sum()),"max_carrier_load":float(y.sum(1).max()),
                "stock_target":float(2*production),"min_stock_slack":float(i
nventory.min()-2*production),
                "scope":"forecast conditional on source-derived mean
response and standard contract caps; no future observed performance
claim"}

```

```

def solve(run):
    f=json.loads((run/"artifacts/facts.json").read_text());d=f["demand"]
    y2,count=count_suppliers(f,d)
    y2,transport2=transfer_fixed_supply(f,y2,d)
    q2=record_plan(f,np.repeat(y2[None,:,:],24,axis=0),d)

    q2["certificates"]={"supplier_selection":count,"transport":transport2}
    print("Q2",count,flush=True)
    y3,proof3=material_plan(f,d,"A_then_C")
    q3=record_plan(f,np.repeat(y3[None,:,:],24,axis=0),d);q3["certificates"]=proof3
    steady,ymax,proofmax=capacity(f)
    immediate=min(steady,(steady+f["initial_inventory_product_equivalent"])/3)
    first,firstproof=material_plan(f,3*immediate-f["initial_inventory_product_equivalent"])
    later,laterproof=material_plan(f,immediate)
    q4=record_plan(f,np.concatenate([first[None,:,:],np.repeat(later[None,:,:],23,axis=0)]),immediate)

```

```

    q4["certificates"]={"steady":proofmax,"first_week":firstproof,"later
_weeks":laterproof}
    q4["steady_capacity"]=steady;q4["steady_increase_percent"]=100*(stea
dy/d-1)
    q4["immediate_increase_percent"]=100*(immediate/d-1)
    # Alternative cap assumptions are sensitivity scenarios, not ground
truth.
    sensitivity=[]
    for scale in (.8,.9,1.,1.1,1.2):
        p,_,proof=capacity(f,scale)
        sensitivity.append({"contract_capacity_scale":scale,"steady_capa
city":p,"immediate_capacity":min(p,(p+2*d)/3),"lp_proof":proof})
        q90=np.array(f["order_cap_q90"])*np.array(f["supply_rate"])
        p,_,proof=capacity(f, cap_override=q90)
        sensitivity.append({"contract_envelope":"positive_order_90th_percent
ile","steady_capacity":p,"scope":"more aggressive order extrapolation,
not certified sustainable capacity","lp_proof":proof})
        p,_,proof=capacity(f, losses=f["carrier_p95_loss"])
        sensitivity.append({"loss_scenario":"carrier marginal 95th
percentiles jointly","steady_capacity":p,"lp_proof":proof})
        write(run/"artifacts/plans.json",{"Q2":q2,"Q3":q3,"Q4":q4})
        write(run/"artifacts/capacity-sensitivity.json",sensitivity)
        print({q:{k:v for k,v in r.items() if k not in
("orders","shipments","certificates","arrivals_product_equivalent","endi
ng_inventory_product_equivalent") } for q,r in
(("Q2",q2),("Q3",q3),("Q4",q4))},flush=True)

if __name__=="__main__":
    ap=argparse.ArgumentParser();ap.add_argument("run",type=Path);solve(
ap.parse_args().run)

```

test_supply.py

```

import unittest
from itertools import combinations

import numpy as np
from solve import capacity,count_suppliers
from verify import greedy_capacity
from prepare import rates

```

```

class SupplyModelTest(unittest.TestCase):
    def fixture(self, cap):
        return
{"supply_cap": cap, "conversion": [.6, .66, .72], "price": [1.2, 1.1, 1.],
    "carrier_mean_loss": [.01, .02, .03, .04, .05, .06, .07, .08]}

    def test_capacity_matches_independent_exchange_construction_with_transport_bottleneck(self):
        f=self.fixture([20000, 20000, 20000])
        actual, _, _=capacity(f); reference, _=greedy_capacity(f["supply_cap"], f["conversion"], f["carrier_mean_loss"])
        self.assertAlmostEqual(actual, reference, places=7)

    def test_supplier_minimum_matches_exhaustive_small_case(self):
        f=self.fixture([4, 5, 3]); demand=10.
        x, proof=count_suppliers(f, demand)
        brute=None
        for n in range(1, 4):
            for ids in combinations(range(3), n):
                cap=[f["supply_cap"][i] if i in ids else 0 for i in range(3)]
                if greedy_capacity(cap, f["conversion"], f["carrier_mean_loss"])[0] >= demand:
                    brute=n; break
            if brute: break
        self.assertEqual(proof["minimum_supplier_count"], brute)

    def test_ratio_mean_includes_failed_order_weeks(self):
        o=np.array([[10., 10., 0.]])
        s=np.array([[10., 0., 0.]])
        self.assertEqual(rates(o, s, "volume_ratio")[0], .5)

if __name__=="__main__": unittest.main()

```

verify.py

"""Independent source accounting, analytic capacity bound and declared stress cases."""

import argparse

import hashlib


```

import json
from pathlib import Path

import numpy as np
from openpyxl import load_workbook
from prepare import write

def greedy_capacity(cap,conversion,loss):
    """Rearrangement bound: best conversion first, then least-loss
    carriers.

    With unrestricted splitting and no other links forbidden, swapping
    two
    allocations improves yield by  $(1/a-1/b)*(\text{loss\_bad}-\text{loss\_good})\geq 0$ .
    """
    remaining=[6000.]*8;total=0.;allocation=np.zeros((len(cap),8))
    for i in sorted(range(len(cap)),key=lambda i:conversion[i]):
        amount=cap[i]
        for j in sorted(range(8),key=lambda j:loss[j]):
            used=min(amount,remaining[j]);amount-=used;remaining[j]-=used
            allocation[i,j]+=used;total+=used*(1-loss[j])/conversion[i]
            if amount<1e-10:break
    return total,allocation

def verify(run):
    f=json.loads((run/"artifacts/facts.json").read_text());plans=json.lo
ads((run/"artifacts/plans.json").read_text())
    wb=load_workbook(next((run/"sources").glob("*附件1*")),read_only=True
,data_only=True)
    sheets=[list(s.iter_rows(min_row=2,max_row=403,values_only=True))
for s in wb];wb.close()
    orders=np.array([r[2:] for r in
sheets[0]],float);supply=np.array([r[2:] for r in sheets[1]],float)
    kinds=[r[1] for r in sheets[0]];conv={"A":.6,"B":.66,"C":.72}[k]
for k in kinds];price={"A":1.2,"B":1.1,"C":1.}[k] for k in kinds]
    cap=np.array([sum(s)/sum(v>0 for v in o) if any(o) else 0 for o,s in
zip(orders,supply)])
    rate=np.array([sum(s)/sum(o) if sum(o)>0 else 0 for o,s in
zip(orders,supply)])

```

```

wb=load_workbook(next((run/"sources").glob("*附件2*")),read_only=True
,data_only=True)
loss=[sum(v for v in row[1:] if v>0)/sum(v>0 for v in row[1:])/100
for row in wb.active.iter_rows(min_row=2,values_only=True)];wb.close()
checks=[];stress={}
def add(name,error,tolerance):checks.append({"name":name,"error":flo
at(error),"tolerance":tolerance,"passed":bool(error<=tolerance)})
add("source-derived supply
envelopes",np.max(np.abs(cap-np.array(f["supply_cap"]))),1e-8)
add("source-derived conditional loss
means",np.max(np.abs(np.array(loss)-f["carrier_mean_loss"])),1e-12)
lower_cap=sorted((v*(1-min(loss))/a for v,a in
zip(cap,conv)),reverse=True)
n=plans["Q2"]["active_suppliers"]
add("n-1 suppliers impossible even at best carrier
loss",max(0.,sum(lower_cap[:n-1])-f["demand"]+1e-6),0.)
for name,p in plans.items():
y=np.array(p["shipments"]);q=np.array(p["orders"]);d=p["producti
on_per_week"]
add(name+" all expected supply
transported",np.max(np.abs(q*rate-y.sum(2))),1e-7)
add(name+" contract capacity",max(0.,np.max(y.sum(2)-cap)),1e-7)
add(name+" carrier capacity",max(0.,np.max(y.sum(1))-6000),1e-7)
add(name+" nonnegative shipments and
orders",max(0.,-min(y.min(),q.min())),1e-8)
inventory=2*f["demand"];inv=[];cost=0.;raw_loss=0.
for t in range(24):
delivered=0.
for i in range(402):
for j in range(8):
delivered+=y[t,i,j]*(1-loss[j])/conv[i]
cost+=y[t,i,j]*price[i];raw_loss+=y[t,i,j]*loss[j]
inventory+=delivered-d;inv.append(inventory)
add(name+" reserve floor",max(0.,2*d-min(inv)),1e-6)
add(name+" inventory reconstruction",np.max(np.abs(np.array(inv)
-p["ending_inventory_product_equivalent"])),1e-6)
add(name+" purchase cost
accounting",abs(cost-p["purchase_cost_index"]),1e-5)
add(name+" physical transport loss
accounting",abs(raw_loss-p["expected_transport_loss"]),1e-5)
scenarios=[]
for multiplier in (.8,.9,1.,1.1):

```

```

        received=(y*multipplier*(1-np.array(loss))[None,None,:]/np.array(conv)[None,:,None]).sum((1,2))
        inv2=2*f["demand"]+np.cumsum(received-d)
        scenarios.append({"supply_multiplier":multipplier,"terminal_inventory":float(inv2[-1]),

"reserve_shortfall_weeks":int((inv2<2*d-1e-6).sum()),
                        "first_production_shortage_week":next((i+1
for i,v in enumerate(inv2) if v<0),None),
                        "fixed_carrier_overload_weeks":int((y.sum(
1)*multipplier>6000+1e-6).any(1).sum()))}
        stress[name]=scenarios
        bound,_=greedy_capacity(cap,conv,loss)
        add("Q4 steady capacity independent rearrangement
construction",abs(bound-plans["Q4"]["steady_capacity"]),1e-6)
        immediate=min(bound,(bound+2*f["demand"])/3)
        add("Q4 first-week inventory upper bound
attained",abs(immediate-plans["Q4"]["production_per_week"]),1e-6)
        report={"passed":all(c["passed"] for c in checks),"checks":checks,

"minimum_count_proof":{"count":n,"best_possible_output_with_n_minus_one":sum(lower_cap[:n-1]),"required":f["demand"],"scope":"conditional on the
declared supplier envelopes"},
        "capacity_bound":{"greedy_steady":bound,"immediate_uniform":
immediate,"proof":"rearrangement for divisible raw material plus
first-week inventory inequality"},
        "stress":stress,"stress_scope":"deterministic response
perturbations, not estimated event probabilities; negative inventory
denotes unmet production, never physical negative stock",
        "plans_sha256":hashlib.sha256((run/"artifacts/plans.json").read_bytes()).hexdigest(),"source_recomputed":True}
        write(run/"artifacts/verification.json",report)
        if not report["passed"]:raise ValueError([c for c in checks if not
c["passed"]])
        print({"checks":len(checks),"passed":True,"capacity_bound":report["c
apacity_bound"],"minimum_count_proof":report["minimum_count_proof"]},flush=True)

if __name__=="__main__":
    ap=argparse.ArgumentParser();ap.add_argument("run",type=Path);verify
(ap.parse_args()).run()

```

workbooks.mjs

```
import fs from 'node:fs/promises';
import path from 'node:path';
import {FileBlob, SpreadsheetFile} from '@oai/artifact-tool';

const run=path.resolve(process.argv[2]);
const mode=process.argv[3] ?? 'preview';
const files=await fs.readdir(path.join(run,'sources'));
const out=path.join(run,'artifacts','submission');
const previews=path.join(run,'tmp','workbooks');
await fs.mkdir(previews,{recursive:true});
if(mode==='fill') await fs.mkdir(out,{recursive:true});
const plans=mode==='fill'?JSON.parse(await
fs.readFile(path.join(run,'artifacts','plans.json'),'utf8')):null;
const facts=mode==='fill'?JSON.parse(await
fs.readFile(path.join(run,'artifacts','facts.json'),'utf8')):null;
for(const tag of ['A','B']) {
  const name=files.find(n=>n.includes('附件'+tag));
  const wb=await SpreadsheetFile.importXlsx(await
FileBlob.load(path.join(run,'sources',name)));
  for(let q=2;q<=4;q++) {
    const sheetName=`问题 ${q} 的 ${tag==='A'? '订购': '转运'} 方案结果`;
    const sheet=wb.worksheets.getItem(sheetName);
    if(mode==='preview') {
      const image=await
wb.render({sheetName,range:'A1:J12',scale:1.5,format:'png'});
      await
fs.writeFile(path.join(previews,`${tag}-Q${q}-before.png`),new
Uint8Array(await image.arrayBuffer()));
      console.log((await wb.inspect({kind:'region',sheetId:sheetName,ran
ge:'A5:J9',maxChars:1200})).ndjson);
      continue;
    }
    const ids=sheet.getRange('A1:A409').values.flat();
    const start=ids.indexOf(facts.ids[0]);
    if(start<0 || !facts.ids.every((id,i)=>ids[start+i]===id)) throw new
Error('supplier row identities differ');
    const data=facts.ids.map((_,i)=>tag==='A'
?plans['Q'+q].orders.map(week=>week[i]>1e-8?week[i]:null)
:plans['Q'+q].shipments.flatMap(week=>week[i].map(v=>v>1e-8?v:null)));
```

```

    const
writable=sheet.getRangeByIndexes(start,1,402,tag==='A'?24:192);
    writable.values=data;
    writable.setNumberFormat('0.000');
    const targetRow=start+plans['Q'+q].orders[0].findIndex(v=>v>1e-6)+1;
    const image=await wb.render({sheetName,range:`A${Math.max(5,targetRow-1)}:J${targetRow+6}`,scale:1.5,format:'png'});
    await fs.writeFile(path.join(previews,`${tag}-Q${q}-after.png`),new
    Uint8Array(await image.arrayBuffer()));
    console.log((await wb.inspect({kind:'region',sheetId:sheetName,range:
    :`A${targetRow}:J${targetRow+2}`,maxChars:1200})).ndjson);
  }
  if(mode==='fill') {
    console.log((await wb.inspect({kind:'match',searchTerm:'#REF!|#DIV/0
    !|#VALUE!|#NAME\\?|#N/A|#NUM!|#NULL!',options:{useRegex:true,maxResults:
    20},maxChars:1600})).ndjson);
    const book=await SpreadsheetFile.exportXlsx(wb);
    await book.save(path.join(out,name.slice(name.indexOf('附件'))));
  }
}

```